EE 4360 Final Project
Design & Simulation of a Tuned PID Controller
Spring, 2025

Sloan Louden-Robins, A05102992

Contents

Section I. Description

This report outlines the process I took to design a Proportional-Integral-Derivative (PID) controller. A PID controller works by continuously calculating an error value as the difference between a desired setpoint and a measured process variable, then applying a correction based on proportional, integral, and derivative terms. These controllers are known for their simplicity and effectiveness in systems where the model is reasonably well-behaved and can be approximated linearly, such as temperature control, motor speed regulation, and position tracking.

Throughout the course of this project, I successfully designed, tuned, and simulated a PID controller to meet specific performance goals, including a 50% improvement in time to peak and maintaining overshoot below 9.48%. Beginning with a mathematical model of the plant, I verified pole placement and transfer function accuracy using both MATLAB and LTSpice. From there, I incrementally added proportional, integral, and derivative terms to shape the system's response. Using a combination of theoretical calculations and simulation-based tuning—including root locus design, Ziegler-Nichols methods, and the Skogestad approach—I was able to derive optimal gain values for all three PID channels. The final implementation demonstrated a 58.9% reduction in time to peak, 39.7% improvement in settling time, and achieved 2% overshoot, all while maintaining strong agreement between MATLAB and LTSpice results. These outcomes show that the controller not only met but exceeded the original design targets.

Section II. Plant Poles

Table 1: Project Parameters	
Parameter	**Goal**
Pole 1	70 rad/s | 11.14Hz
Pole 2	90 rad/s | 14.32Hz
Pole 3	100 rad/s | 15.92Hz
Tp % Improvement	50%
%OS	9.48%

Table 1. Table of transfer function parameters.

Simulation in Matlab:

The formulas below were used to determine the numerator of the transfer function provided. The numerator is the product of the real values of all of the poles.

$$[1] \quad G(s) = \frac{(70)(90)(100)}{(s+70)(s+90)(s+100)}$$

$$[2] \quad G(s) = \frac{630{,}000}{(s+70)(s+90)(s+100)}$$

The poles are given in radians per second, to convert them to hertz equation [3] is used.

$$[3] \quad Hz = \frac{rad/s}{2\pi}$$

Resulting in pole 1 at 11.14Hz, pole 2 at 14.32Hz and pole 3 at 15.92Hz as seen in Table 1.

Shown below is the code used for the Matlab simulation.

```
s = tf('s');
G = (70)*(90)*(100)/((s+70)*(s+90)*(s+100));
h = bodeplot(G);
p = getoptions(h);
p.FreqUnits = ‘Hz’;
setoptions(h,p);
```

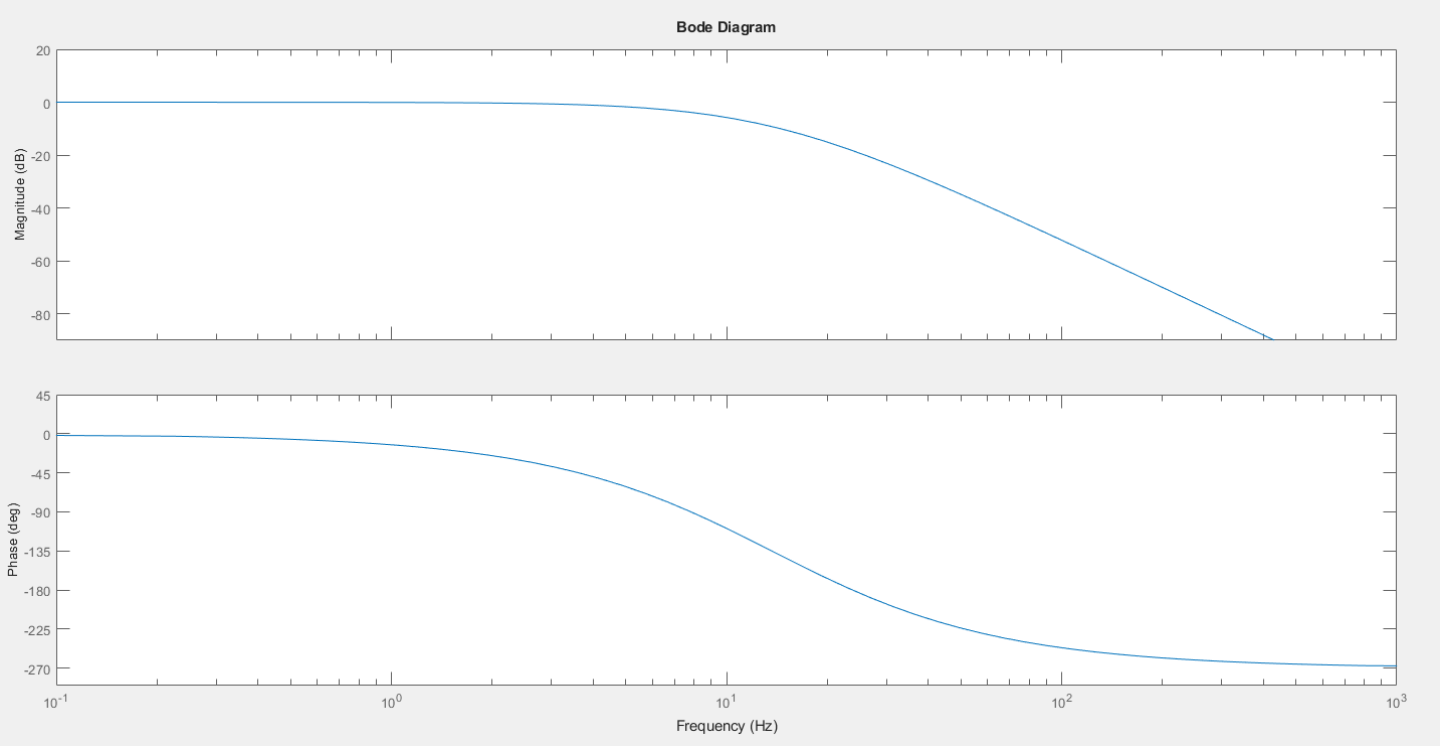

Figure 1. Transfer function G(s) bode plot simulated in Matlab.

The 3 frequencies that are listed below were determined for data tip placements where data tip 1 is half of the lowest frequency, data tip 2 is found using equation [4] and data tip 3 is placed at twice the highest frequency. These cursor placements are seen in figure 2 showing the corresponding magnitude values.

1. 35 rad/s = 5.57Hz
2. 95 rad/s = 15.11Hz
3. 200 rad/s = 31.83Hz

$$[4] \quad \frac{(P3-P2)}{2}$$

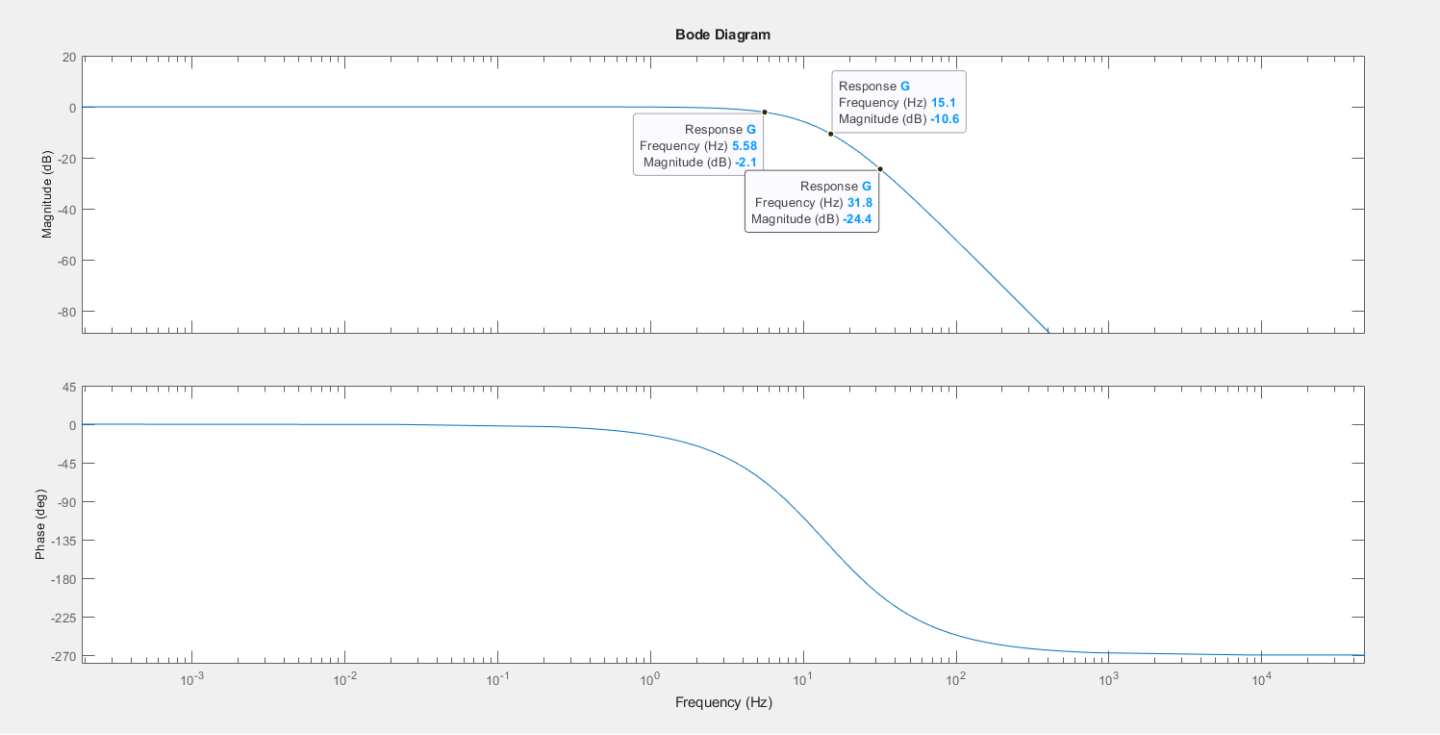

Figure 2. MatLab bode plot of transfer function with data tip cursors placed at 5.58Hz with magnitude of -2.1dB, the second cursor placed at 15.1Hz with a magnitude of -10.6dB, and the third cursor at 31.8Hz with a magnitude of -24.4dB.

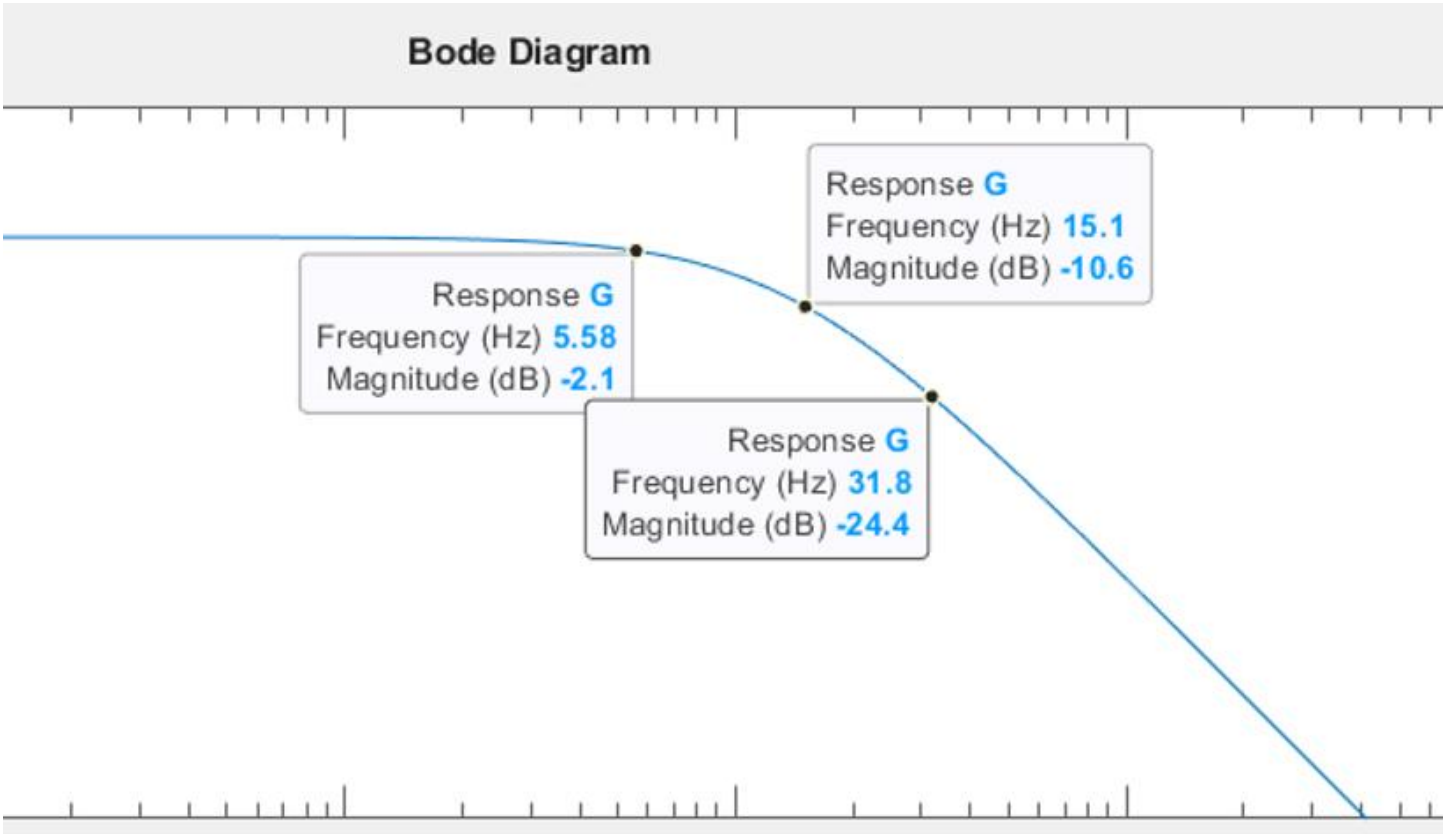

Figure 3. Enlarged Matlab Bode plot of plant showing the 3 data tip points and their corresponding magnitudes.

Simulation In LTSpice

For the simulated circuit a resistor value of 1KΩ was placed at each circuit and this value was used to determine the required capacitance value as calculated in equations [5].

$$[5] \quad C1 = \frac{1}{1{,}000\Omega(70)} = 14.3\mu F$$

$$[5] \quad C2 = \frac{1}{1{,}000\Omega(90)} = 11.1\mu F$$

$$[5] \quad C3 = \frac{1}{1{,}000\Omega(100)} = 10\mu F$$

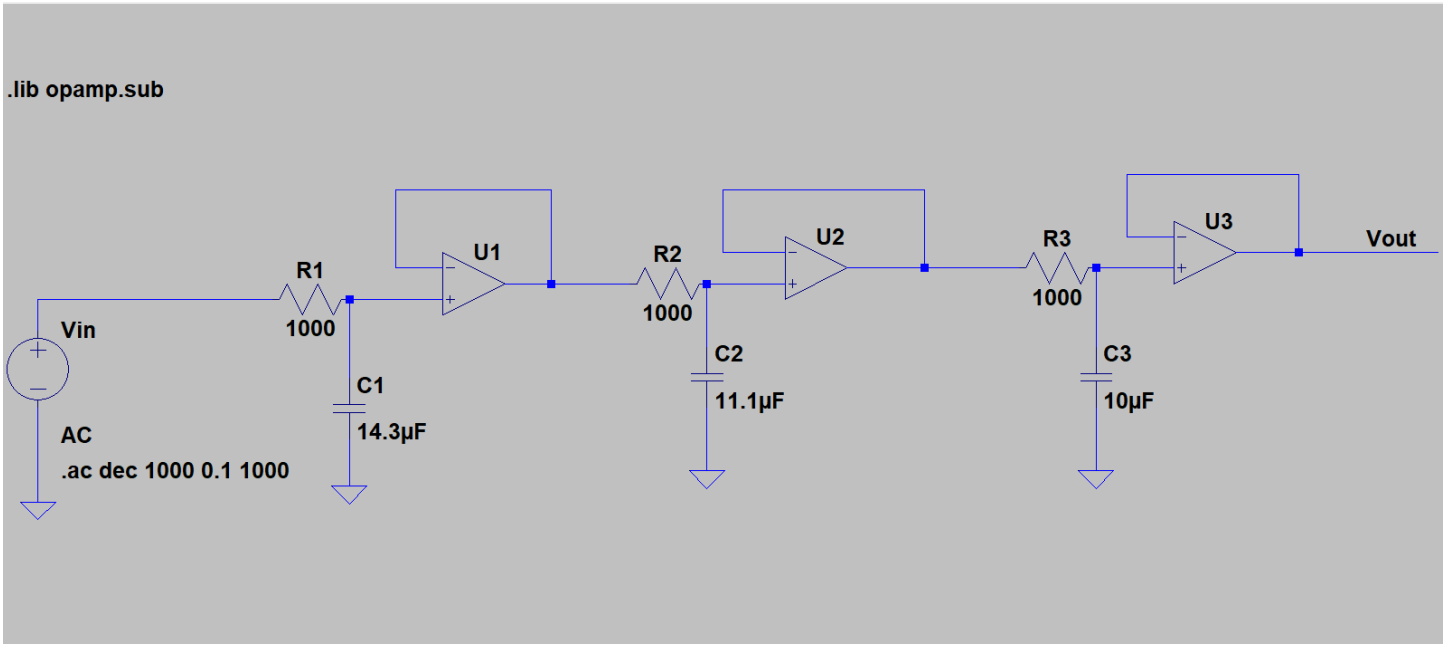

Figure 4. Circuit schematic of plant 17 simulated in LTSpice with 3 op amp functions connected sequentially with a 1000K resistor in series with a proportional capacitor value determined using the formulas shown below connected at the positive input terminal and the negative terminal connected to the output. The Circuit is connected to an AC voltage source that starts at 0.1Hz and sweeps to 1KHz with 1000points/decade.

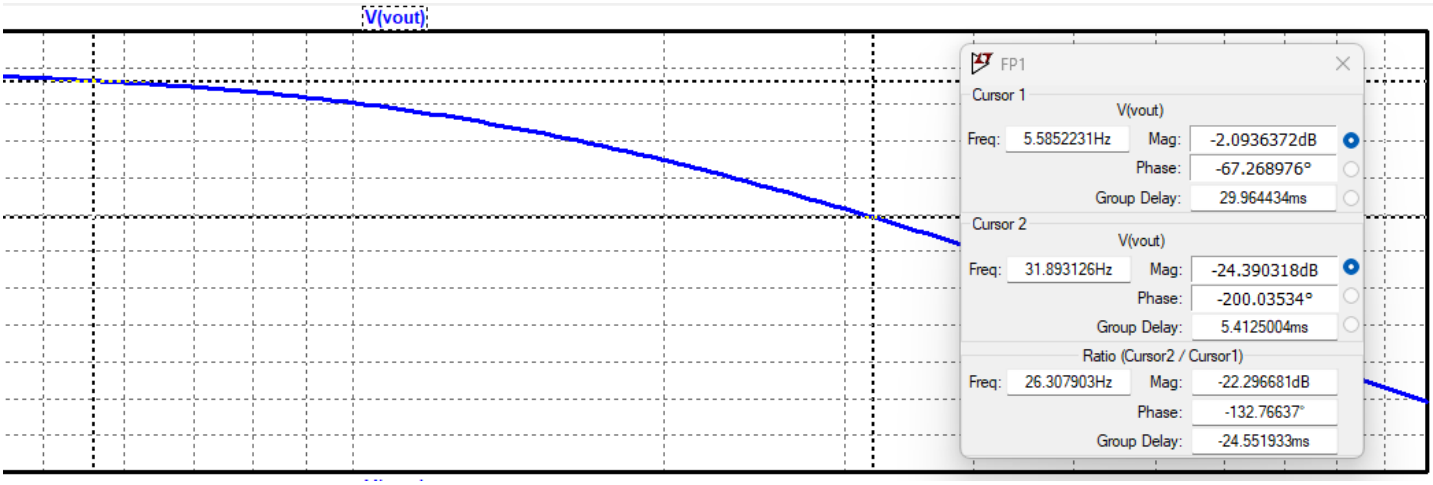

Figure 5a. Bode plot of transfer function simulated in LTSpice with two cursors placed at p1 and p3 where p1 is the lowest frequency measurement at 5.58Hz with a magnitude of -2.09dB and the second cursor placed p3 which is the highest frequency measurement at 31.89Hz with a magnitude of -24.39dB.

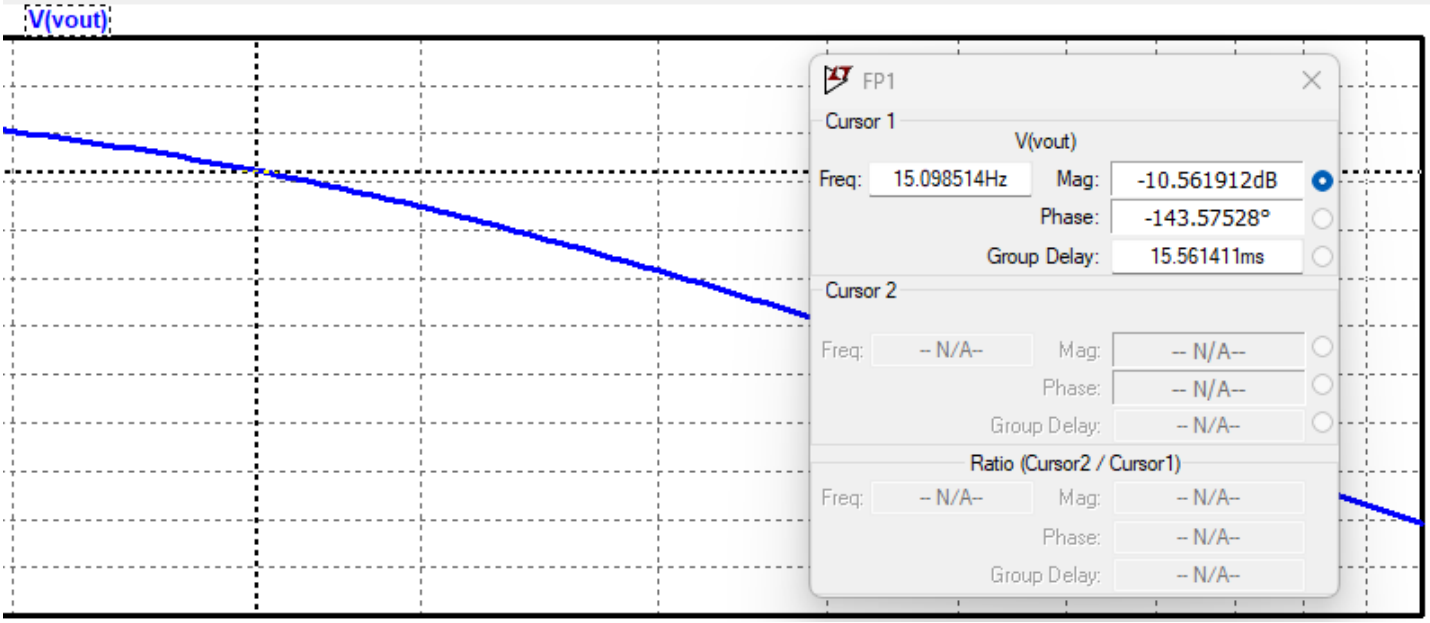

Figure 5b. Bode plot of transfer function simulated in LTSpice with 1 cursor placed p2 which is the middle frequency measurement of 15.09Hz with a magnitude of -10.56dB.

The magnitudes measured from both Matlab and LTSpice at the three critical frequencies show little deviation, ranging from 0.04% to 0.48%, which is within acceptable limits to validate that in both simulations the poles are in the correct placement and the transfer function behaves the same. A tabular description of the results are listed below in table 2.

Table 2: Magnitude Comparison, Matlab Vs. LTSpice			
Frequency, Hz	**Matlab Magnitude, dB**	**Simulation Magnitude, dB**	**% Δ**
5.58Hz (35 rad/s)	-2.1dB	-2.09dB	0.48%
15.1Hz (95 rad/s)	-10.6dB	-10.56dB	0.38%
31.8Hz (200 rad/s)	-24.4dB	-24.39dB	0.04%

Table 2. Table of three frequency points with their measured magnitudes in LTSpice and Matlab and the percent difference between the values.

Section III. Step Response

LTSpice Simulation Step Response:

Plant 17 is simulated in LTSpice with negative feedback driving the plant and the gain, K=1. Equation [6] below is used to calculate the %OS of the plant system with K = 1 (unity gain).

$$[6] \quad \%OS = \frac{C_{max} - C_{final}}{C_{final}} \times 100$$

Applying equation [6], we find the percent overshoot for the plant in LTSpice is 13.4%

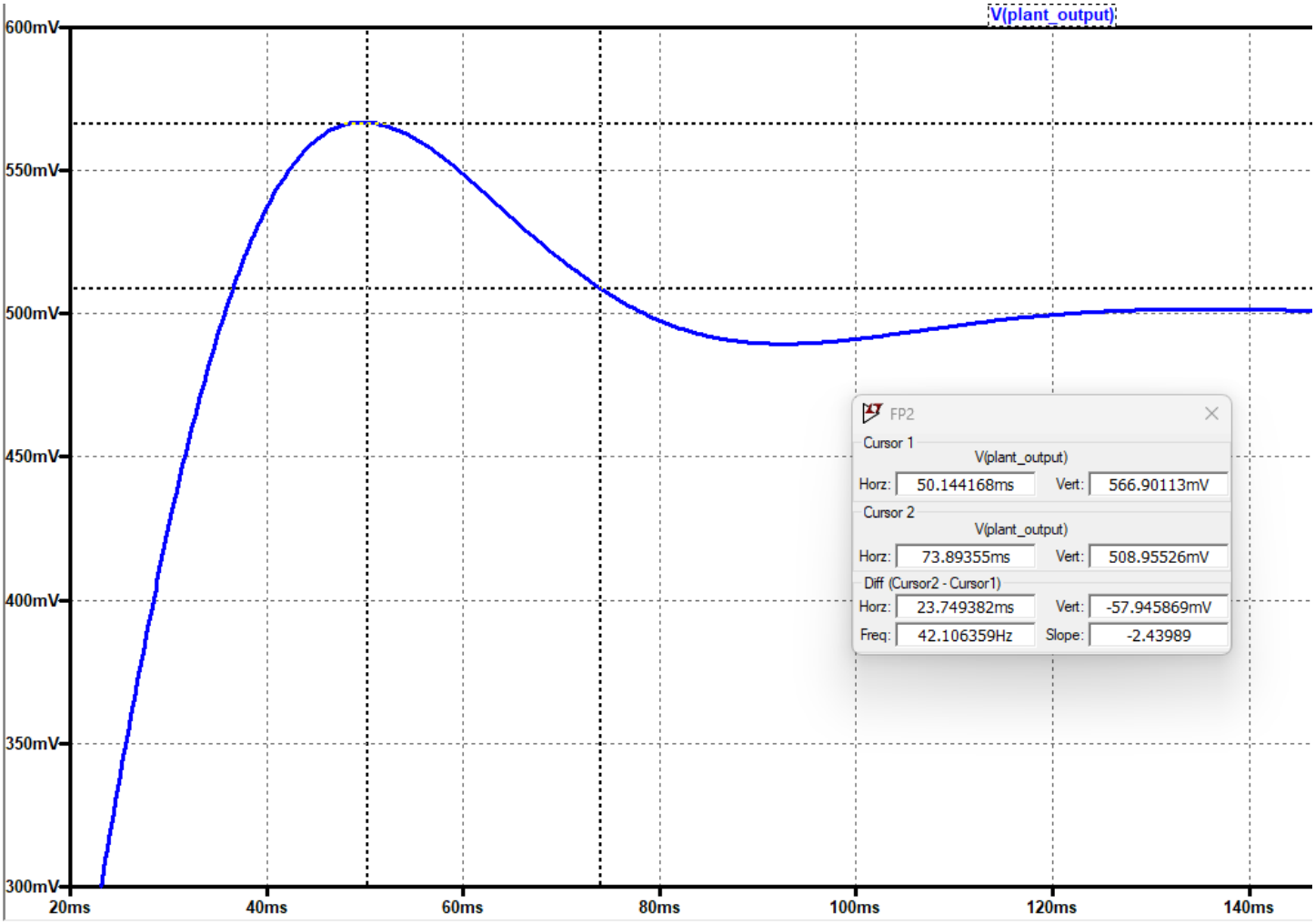

Figure 6. Step response of plant system output simulated in LTSpice with one cursor placed at the peak value of 566.9mV and time of 50.1ms. The second cursor is placed at 508.9mV which is within 2% of the final value of 500mV indicating the settle time of 73.89ms.

Matlab Step Response:

Shown below is the Matlab code used to simulate the step response of plant 17 with unity gain.

```
s = tf('s');
G = (70)*(90)*(100)/((s+70)*(s+90)*(s+100));
K = 1;
sys_cl = feedback(K*G,1);
step(sys_cl,1);
axis([0 0.3 0 0.6]);
```

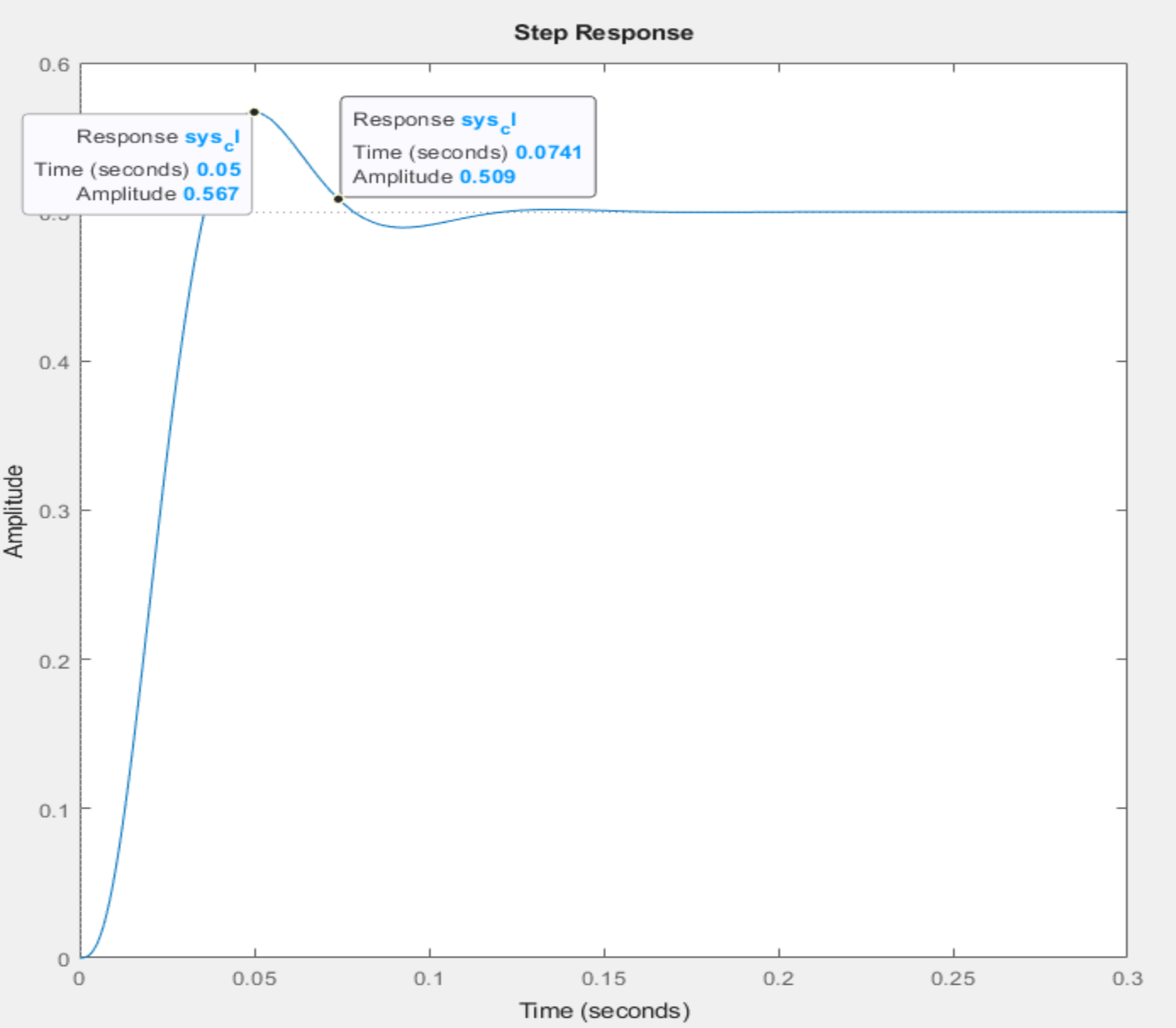

Figure 7. Step response of plant system in matlab with one cursor placed at the peak value of 567mV and time of 50ms. The second cursor is placed at 509mV which is within 2% of the final value of 500mV indicating the settle time of 74.1ms.

The Matlab simulation resulted in a peak time of 50.0ms and a settling time of 74.1ms, with a %OS of 13.4% and a final value of 500 mV. Similarly, the LTSpice simulation showed a peak time of 50.1ms, a settling time of 73.89ms, a %OS of 13.38%, and a final value of 500 mV. These results show consistency between the two simulation environments, with negligible differences: 0.2% in peak time, 0.3% in settle time, 0.1% in %OS and an exact match in FV as reported in table 3. These results verify the system responds as expected when input with a step response with unity gain and negative feedback.

Table 3: Step Response Comparison, Matlab vs. LTSpice			
Parameter	**Matlab**	**LTSpice**	**% Δ**
Tp	50ms	50.1ms	0.2%
Ts	74.1ms	73.89ms	0.3%
%OS	13.4%	13.38%	0.1%
FV	500mV	500mV	0%

Table 3. Table comparing time to peak, time to settle, final value and %OS in Matlab and LTSpice with their percent differences.

Section IV. Plant Verification

ζ is calculated to be equal to 0.54 by equation [7] with max %OS = 13.4% as found in section III.

$$[7] \quad \zeta = \frac{-\ln(\frac{\%OS}{100})}{\sqrt{\pi^2 - \ln^2(\frac{\%OS}{100})}}$$

Matlab Simulation:

The code used for the simulation is shown below, with the natural frequency (wn) set to a value large enough to not interfere, in this case I chose 500 as that's 5x larger than the largest pole's frequency of 100.

```
s = tf('s');
G = (70)*(90)*(100)/((s+70)*(s+90)*(s+100));
zeta = 0.54;
wn = 500;
rlocus(G);
sgrid(zeta,wn);
axis([-110 5 -80 80]);
```

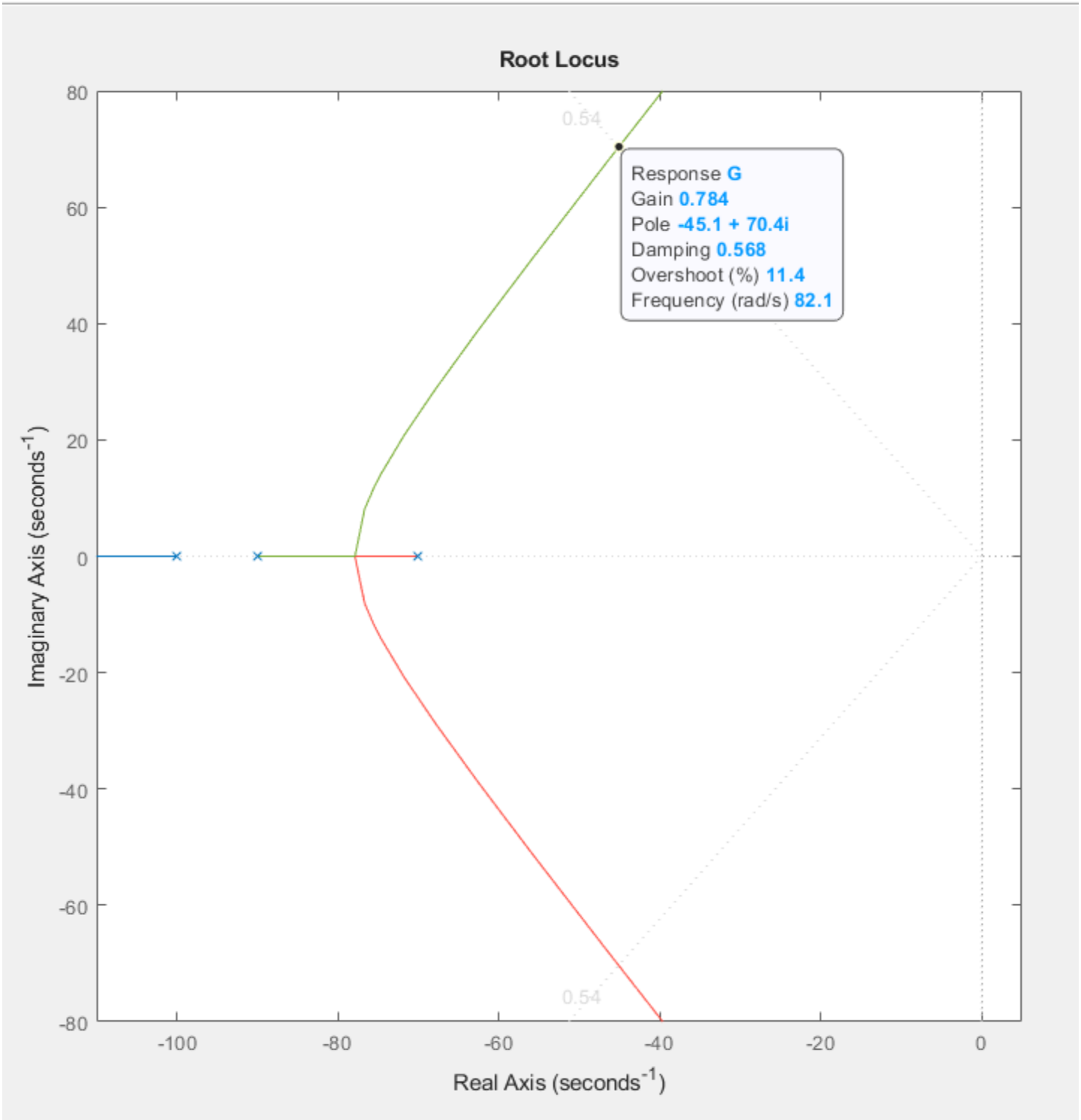

Figure 8. Matlab simulation to find Root Locus intersection of plant 17. Intersects at ζ = 0.568, K = 0.784, with poles at -45.1 ± j70.4

Using the same code from section III, the gain (K), is adjusted based on the gain found at the root locus intersection point (K = 1 → K = 0.784) and the step response for the plant is modeled as seen below in figure 9. The time to peak is found to be 52.5ms, time to settle is 81.5ms, the magnitude of the peak is 482mV and the final value is 439mV.

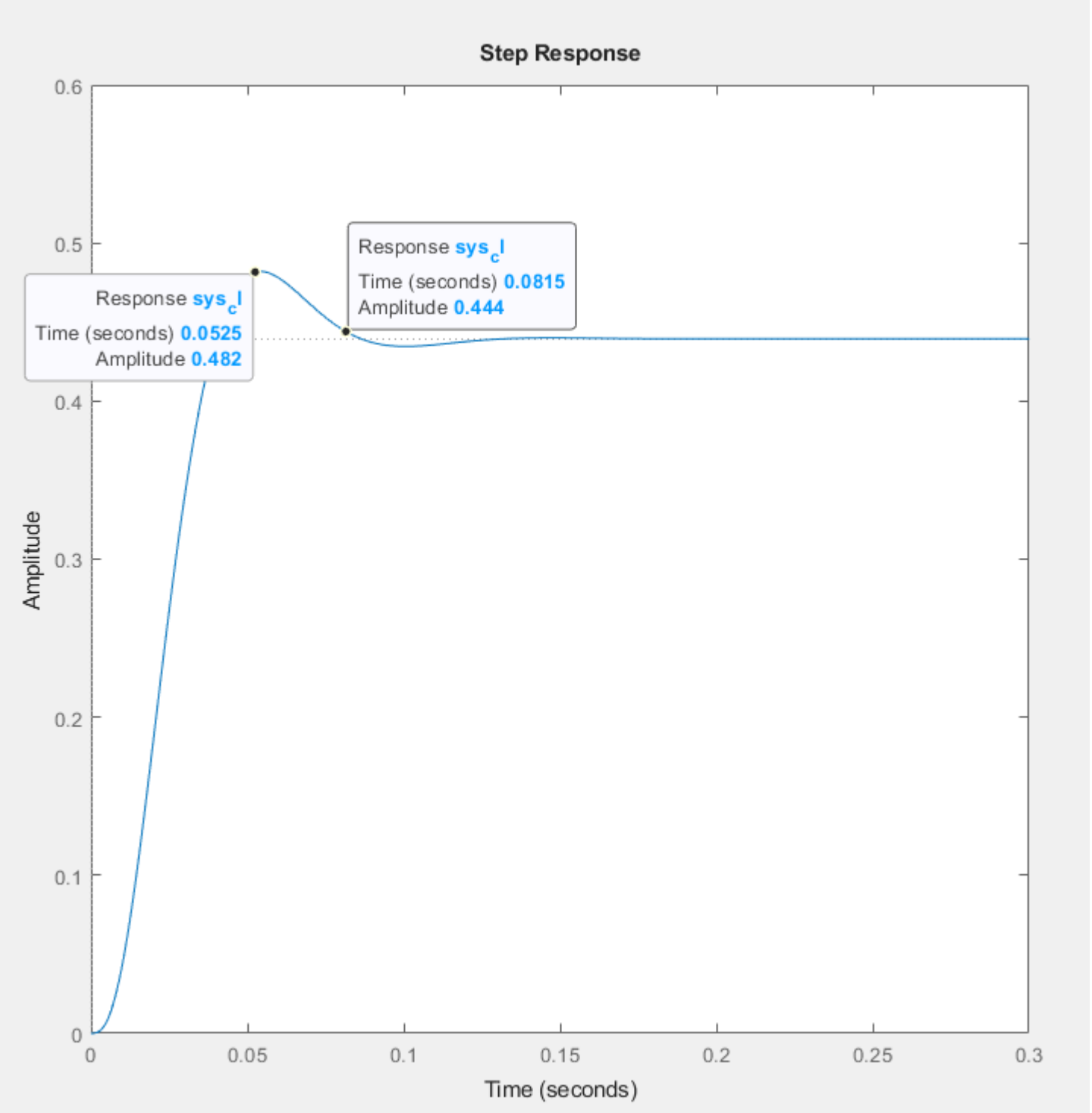

Figure 9. Matlab step response with gain, K = 0.784. Time to peak is 52.5ms, time to settle is 81.5ms, %OS is 11.4% and the final value is 439mV.

LTSpice Simulation:

The step response for the plant is modeled in LTSpice as shown below in figure 10, with the resulting step response graph as seen in figure 11. The time to peak is found to be 52.62ms, time to settle is 81.22ms, the magnitude of the peak is 481.7mV and the final value is 439mV.

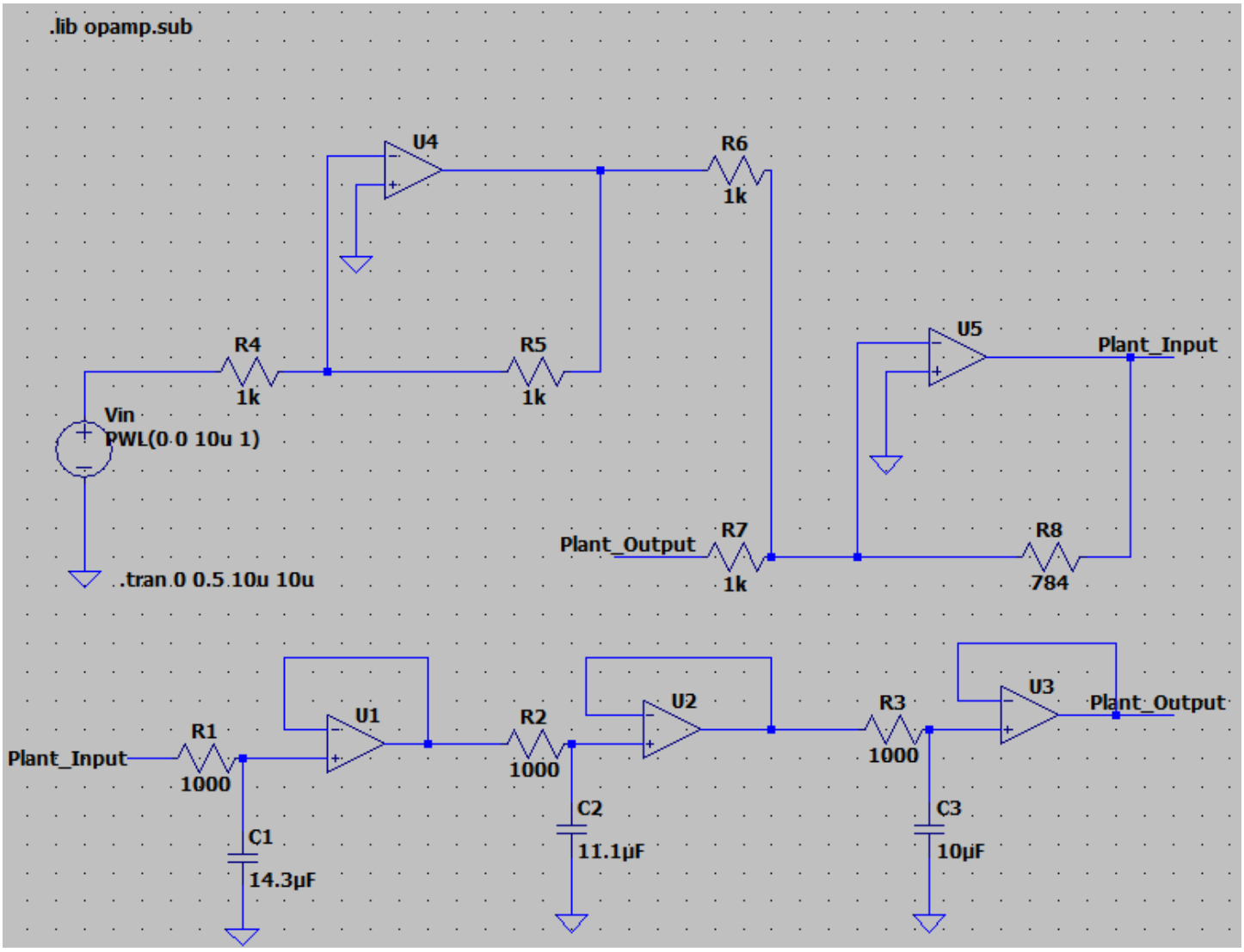

Figure 10. Plant 17 schematic for step response with gain, K = 0.784.

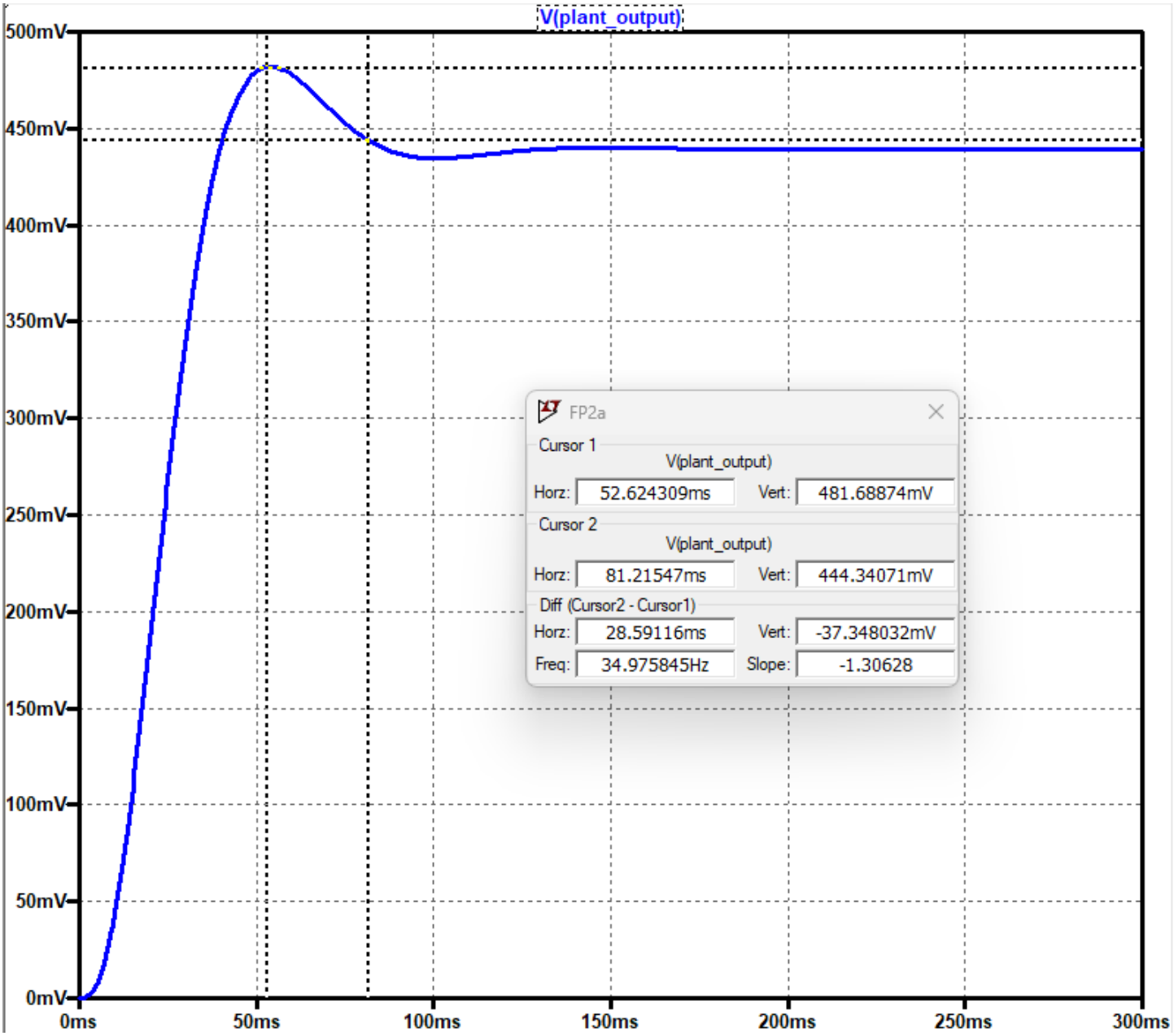

Figure 11. LTSpice simulation on plant 17 step response with gain K = 0.784. Two cursors are placed at the peak value of 481.69mV and the second cursor is placed at 444.34mV which is within 2% of the final value of 439mV.

Table 4: Performance Comparison, Matlab vs. LTSpice Operating at ζ = 0.54 and K = 0.784			
Parameter	**Matlab**	**LTSpice**	**% Δ**
Tp	52.5ms	52.6ms	0.2%
Ts	81.5ms	81.2ms	0.4%
%OS	11.4	11.4	0%
FV	439mV	439mV	0%

Table 4. *Table showing step response for G(s) operating at ζ = 0.54 and K = 0.784 and comparing time to peak, time to settle, final value and %OS in Matlab and LTSpice with their percent differences.*

Section V. PD Calculations & Matlab Verification

Matlab code for adding the integrator to the system:

```
s = tf('s');
G = ((70)*(90)*(100))*(s+7)/(s*(s+70)*(s+90)*(s+100));
rlocus(G);
zeta = 0.54;
wn = 300;
sgrid(zeta,wn);
axis([-100 5 -75 75]);
```

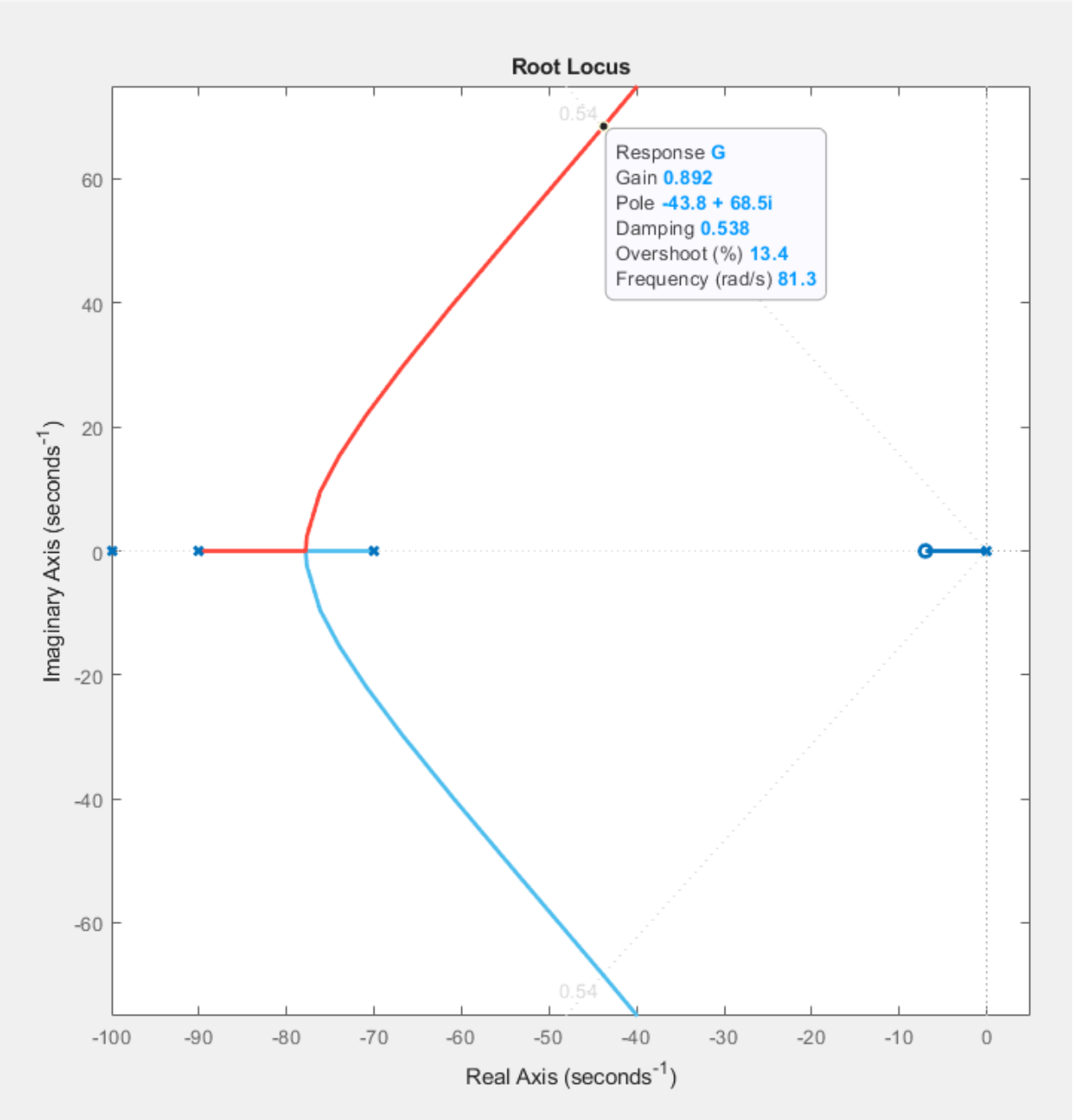

Figure 12. root locus. Note that the root locus intersects the ζ line at -43.8 ± j68.5 but used to intersect at –45.1 ± j70.4, meaning the system is a little slower after adding the integrator. Gain K has increased from 0.784 to 0.892.

Matlab code for system step response with added integrator:

```
s = tf('s');
G = (((70)*(90)*(100))*(s+7))/(s*(s+70)*(s+90)*(s+100));
zeta = 0.54;
wn = 300;
K = 0.892;
sys_cl = feedback(K*G,1);
step(sys_cl,1);
axis([0 1 0 1.2]);
```

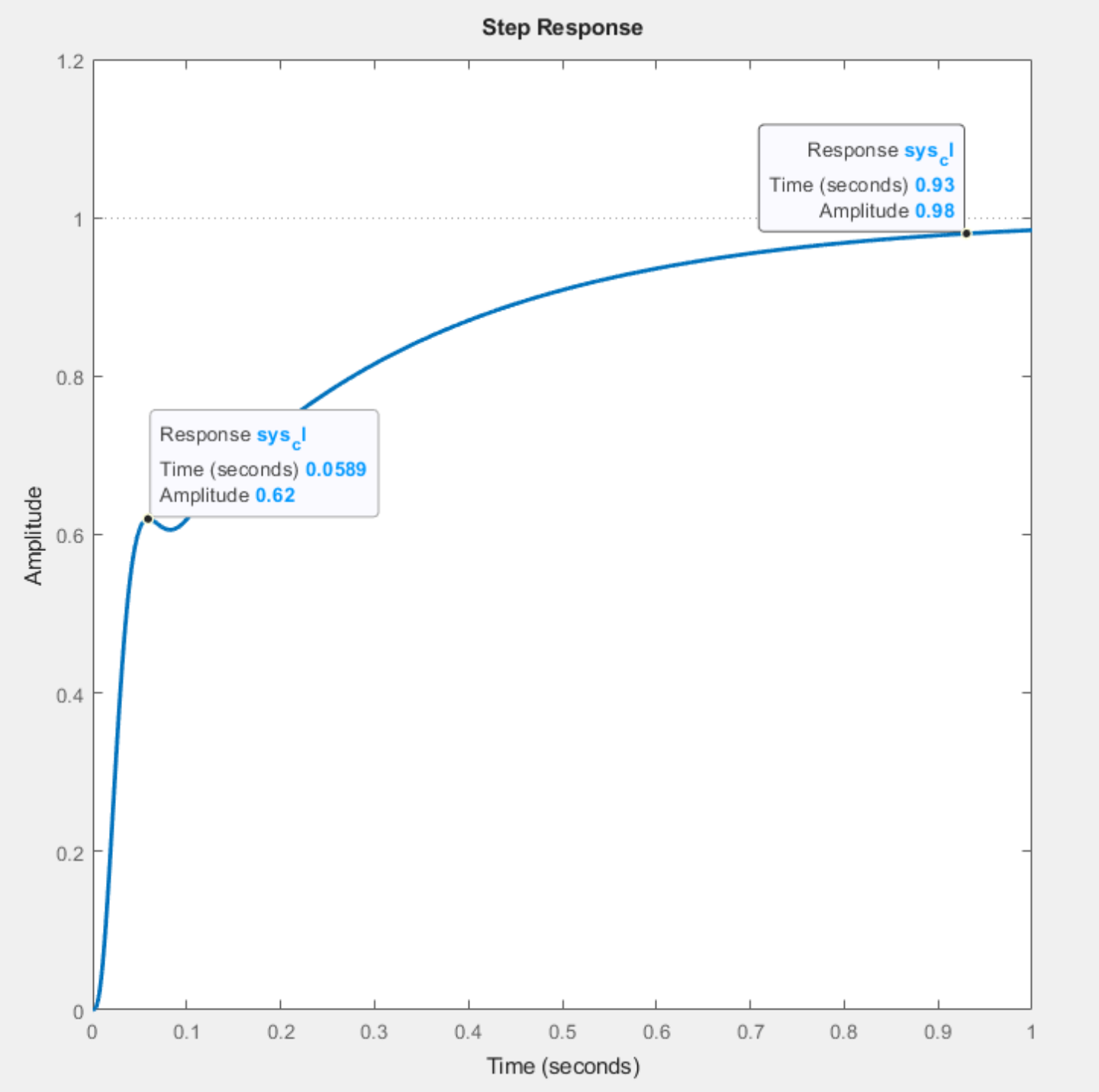

Figure 13. Step response with PI compensation, zero at -7 rad/s, and gain K=0.892.

Performance Comparison, P vs. PI Compensation at *ζ = 0.54 and PI Zero at -7.0 rad/s*			
Parameter	**P Only**	**PI**	**% Δ**
TP	52.5ms	58.9ms	+ 11.49%
Pole Location	–45.1 ± j70.4	*-43.8 ± j68.5*	(Re) -2.92% / (Im) 2.74%
TS	81.5ms	930ms	N/A
% OS	11.4%	0%	N/A
FV	439mV	980mV	N/A
K (Gain)	0.784	0.892	N/A

Table 5. Performance comparison with PI zero at -7.0rad/s.

Matlab code for Root Locus with PI zero farther from origin:

```
s = tf('s');
G = (((70)*(90)*(100))*(s+9))/(s*(s+70)*(s+90)*(s+100));
zeta = 0.54;
wn = 300;
rlocus(G);
sgrid(zeta,wn);
axis([-100 5 -75 75]);
```

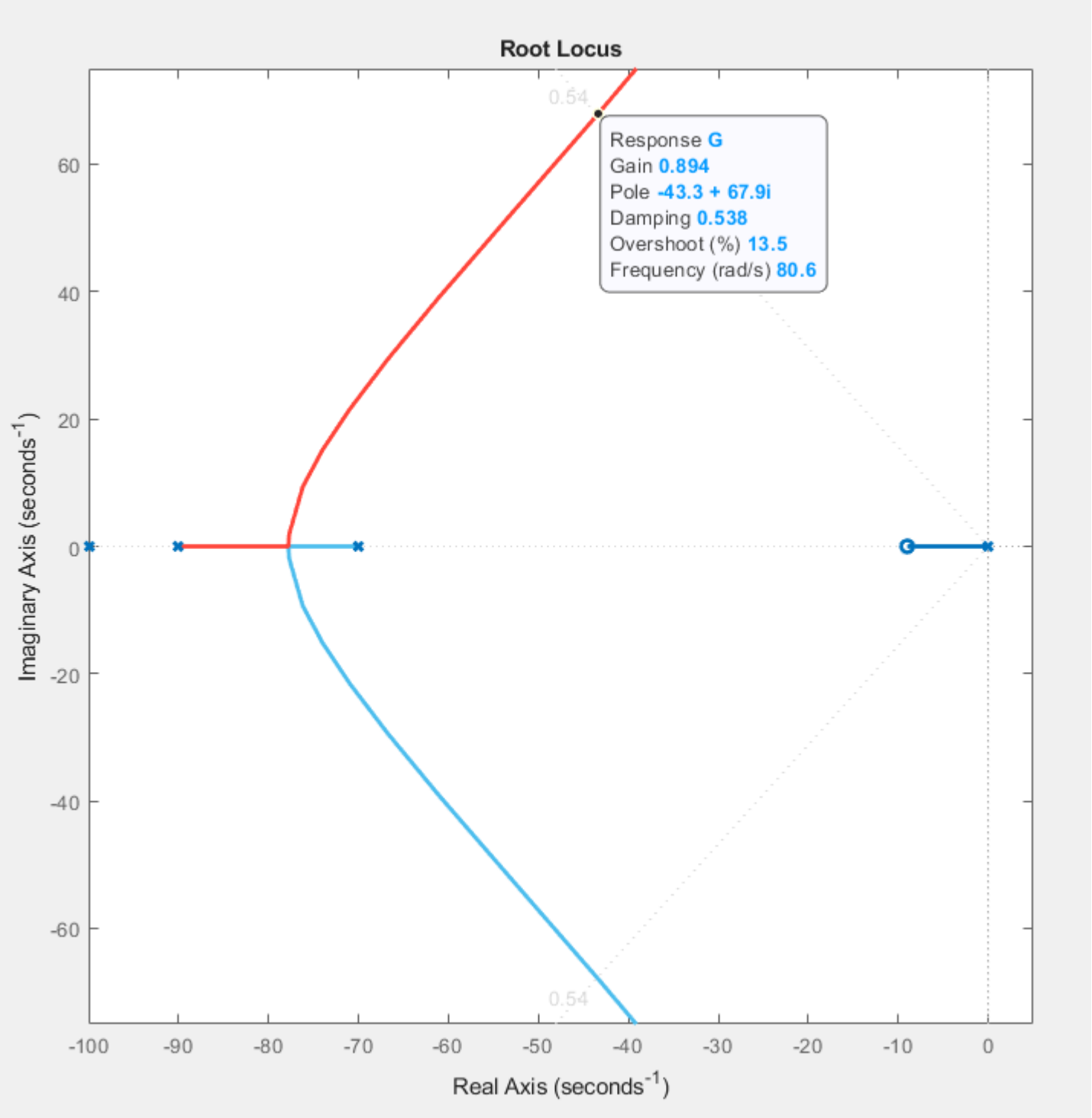

Figure 14. Root locus with PI zero at -9 rad/s. Gain K changes from 0.892 to 0.894.

Matlab code for system step response with PI zero farther from origin:

```
s = tf('s');
G = (((70)*(90)*(100))*(s+9))/(s*(s+70)*(s+90)*(s+100));
zeta = 0.54;
wn = 300;
K = 0.892;
sys_cl = feedback(K*G,1);
step(sys_cl,1);
axis([0 1 0 1.2]);
```

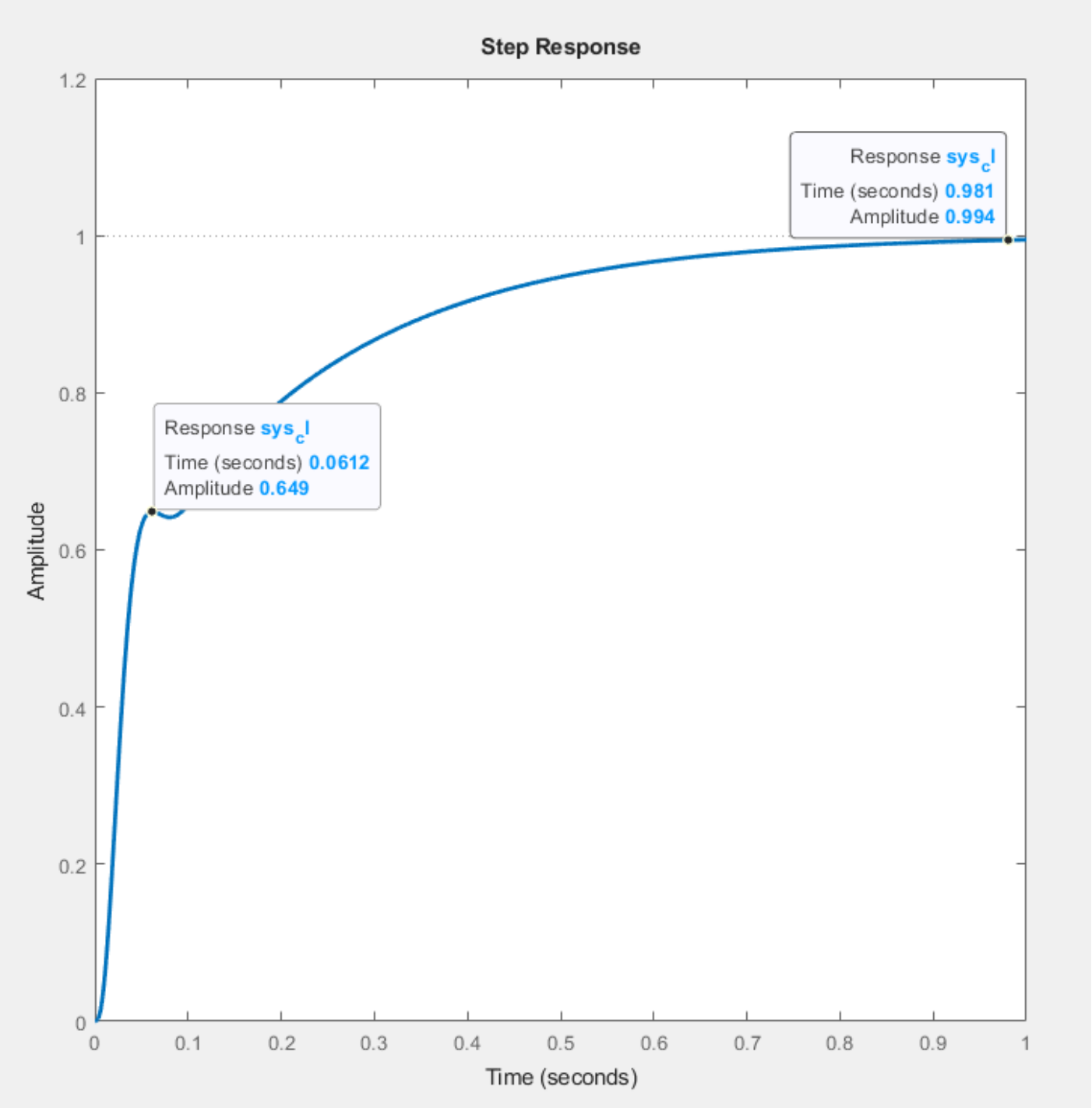

Figure 15. Matlab step response with PI compensation zero at -9 rad/s and gain K=0.894.

Table 6: Performance Comparison, P vs. PI Compensation at *ζ = 0.54 and PI Zero at -9.0 rad/s*			
Parameter	**P Only**	**PI**	**% Δ**
TP	52.5ms	61.2ms	+ 15.30%
Pole Location	–45.1 ± j70.4	*-43.3 ± j67.9*	(Re) 4.07% / (Im) 3.62%
TS	81.5ms	930ms	N/A
% OS	11.4%	0%	N/A
FV	439mV	981mV	N/A
K (Gain)	0.784	0.894	N/A

Table 6: Performance comparison with PI zero at -7.0rad/s.

Derivative Channel Design Calculations:
For the original plant the dominate poles were located at –45.1 ± j70.4 rad/s, the target goal is to improve the time to peak (Tp) by 50%. To achieve this the dominant pole pair has to be moved further away from the origin. The formula below is used to find Tp, however it is noted that this value will differ slightly from the actual measured value in Matlab because the plant is a third order system not a second order.

$$[8] \quad Tp = \frac{\pi}{|Im|} = \frac{\pi}{|70.4|} = 44.6ms$$

The measured Tp as shown in table 6 is 52.5ms, so to improve this by 50% our goal is to determine the dominate poles that result in a Tp = 26.25ms. The equations below demonstrate how to find the imaginary component (equation 9) and the real component (equation 10) of the new dominate pole.

$$[9] \quad |Im| = \frac{\pi}{|Tp|} = \frac{\pi}{|0.02625|} = 119.68\, rad/s$$

$$[10] \quad |Re| = \frac{|Im\ (new)|}{|Im\ (original)|}|Re| = \frac{|119.68|}{|70.4|}|45.1| = 76.67\, rad/s$$

From the equations 9 and 10 the desired pole location for 50% improvement is -76.67 ± j119.68 rad/s. This point on the root locus has to be an odd multiple of ± 180°. Using Matlab the existing angle at that point is calculated using the sum of the angles of the poles and zeros.

The code for the pole-zero plot is shown below:

```
s = tf('s');
G = (((70)*(90)*(100))*(s+9))/(s*(s+70)*(s+90)*(s+100));
zeta = 0.54;
wn = 300;
pzmap(G);
axis([-100 5 -5 125]);
```

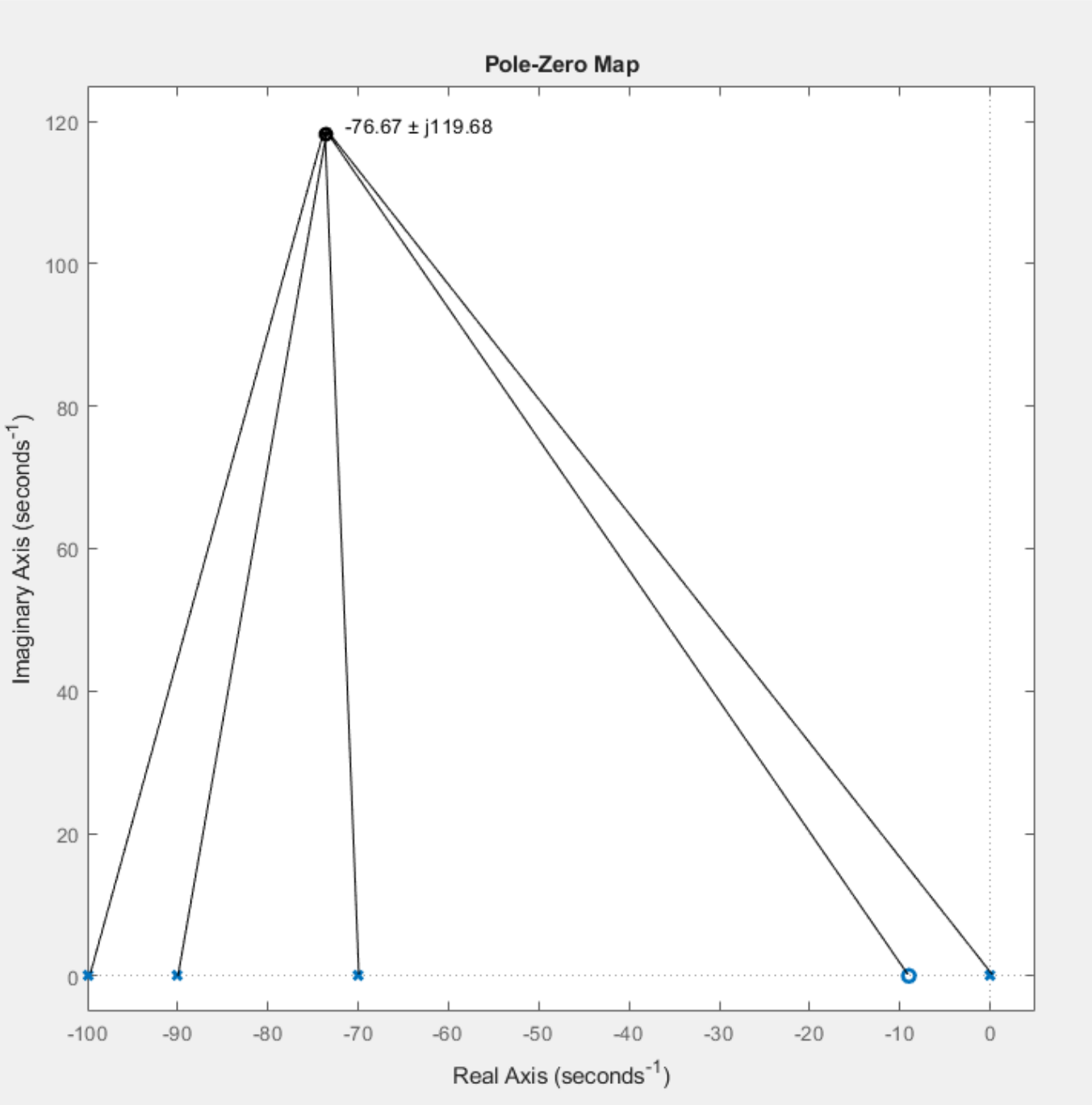

Figure 16a. Matlab pole-zero map used to calculate frequency of D-channel compensating zero.

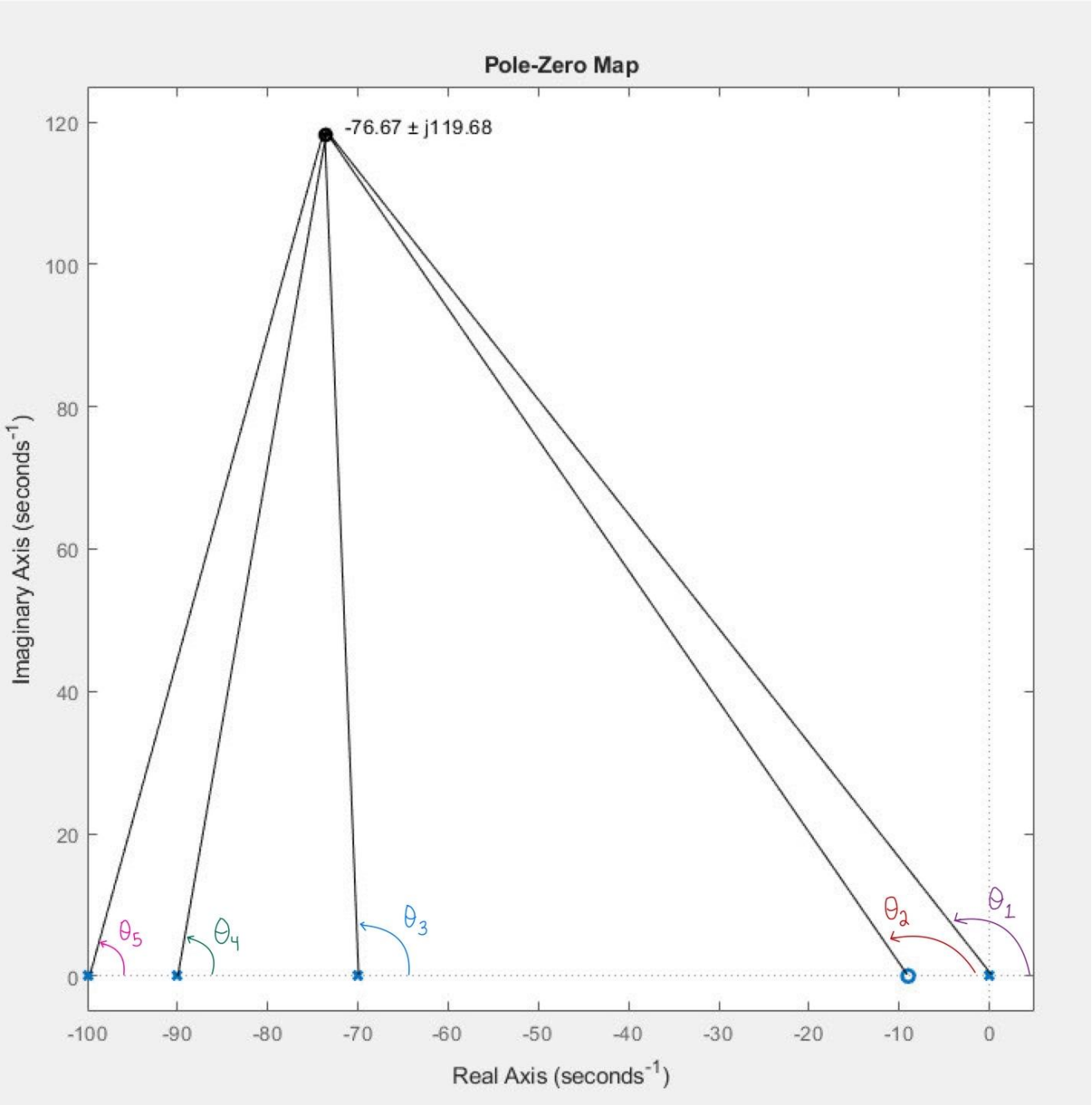

Figure 16b. Matlab pole-zero map with θ_{1-5} labeled.

$$[11] \quad 180^0 - \tan^{-1}\left(\frac{119.68}{76.67}\right) = 122.65^0$$

$$180^0 - \tan^{-1}\left(\frac{119.68}{76.67-10}\right) = 119.12^0$$

$$180^0 - \tan^{-1}\left(\frac{119.68}{76.67-70}\right) = 93.19^0$$

$$\tan^{-1}\left(\frac{119.68}{90-76.67}\right) = 83.64^0$$

$$\tan^{-1}\left(\frac{119.68}{100-76.67}\right) = 78.97^0$$

$$[12] \quad \sum\theta_z - \sum\theta_P = 119.12^0 - 378.45^0 = -259.33^0$$
$$-259.33^0 + 180^0 = -79.33^0$$
$$-259.33^0 + 79.33^0 = -180^0$$

The poles and zeros angles were calculated using the equations shown in 11, taking those angles, the sum of poles and zeros are calculated and the value of the added angle needed to bring the total to be an odd multiple of 180^0 is determined by using equations in 12. This angle determines the location of the compensating zero, as shown below in figure 17, using equations 13 and 14.

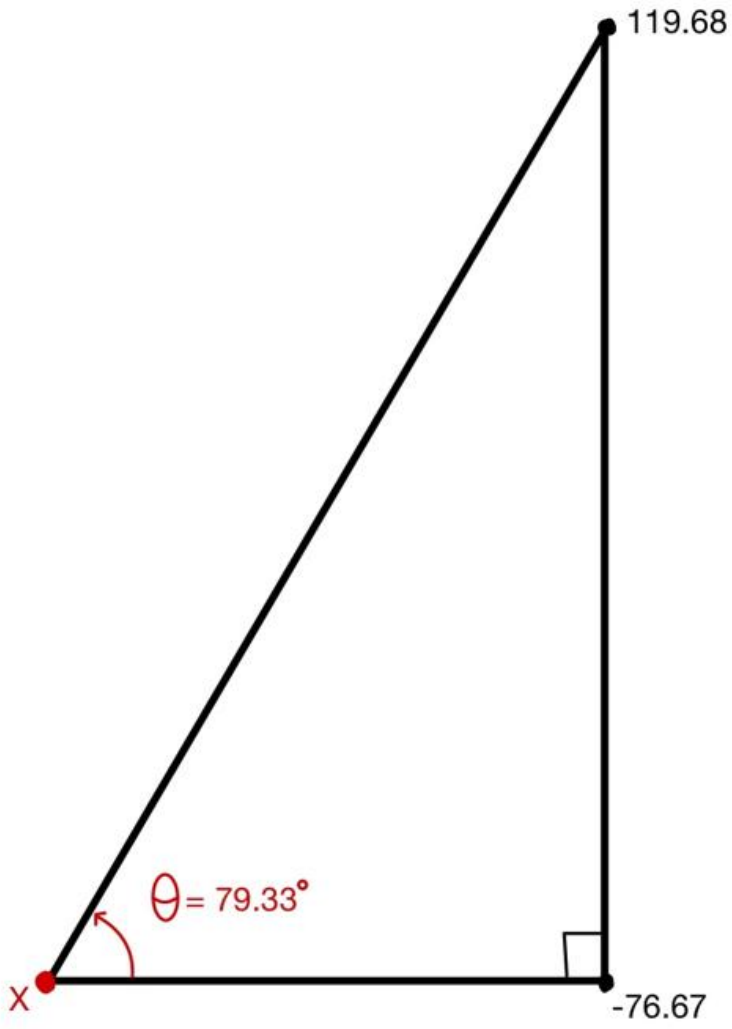

Figure 17. Right triangle used for calculating location of compensating zero.

$$[13] \quad X = \frac{119.68}{\tan(79.33^0)} = 22.55$$

$$[14] \quad Zero\ Location\ (X) = -76.67 - 22.55 = -99.22\ rad/s$$

Verifying the new pole location is done in Matlab using the code below:

```
s = tf('s');
G = (((70)*(90)*(100))*(s+9)*(s+99.22))/(s*(s+70)*(s+90)*(s+100));
zeta = 0.54;
wn = 300;
rlocus(G);
sgrid(zeta,wn);
axis([-120 5 -150 150]);
```

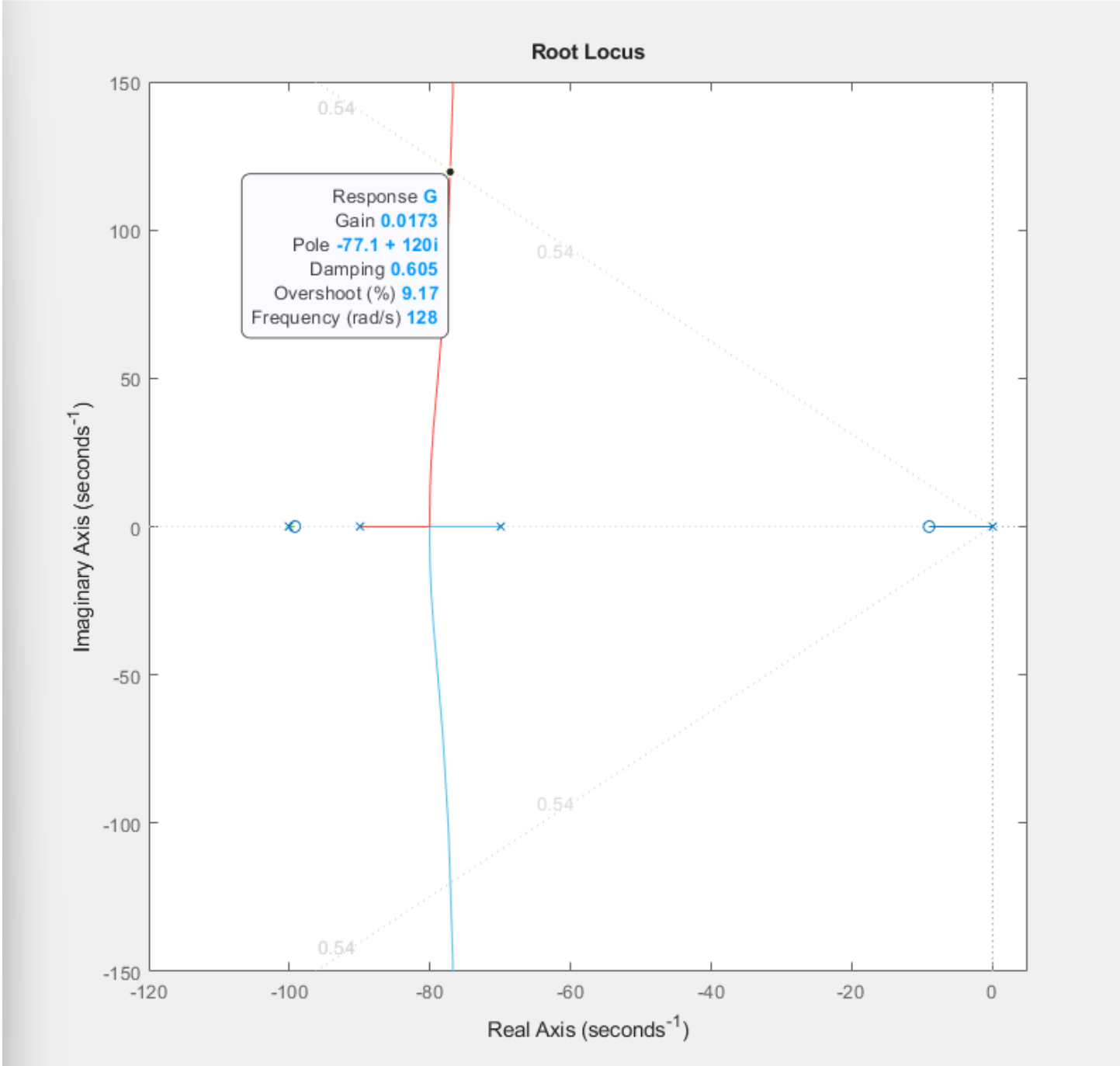

Figure 18. Root locus with compensating zero at -77.1 ± j120 rad/s close to the expected -76.67 ± j119.68 rad/s with gain K = 0.0173, 9.17% overshoot.

Matlab code for step response of PID:

```
s = tf('s');
G = (((70)*(90)*(100))*(s+9)*(s+99.22))/(s*(s+70)*(s+90)*(s+100));
zeta = 0.54;
wn = 300;
K = 0.0173;
sys_cl = feedback(K*G,1);
step(sys_cl,1);
axis([0 1 0 1.2]);
```

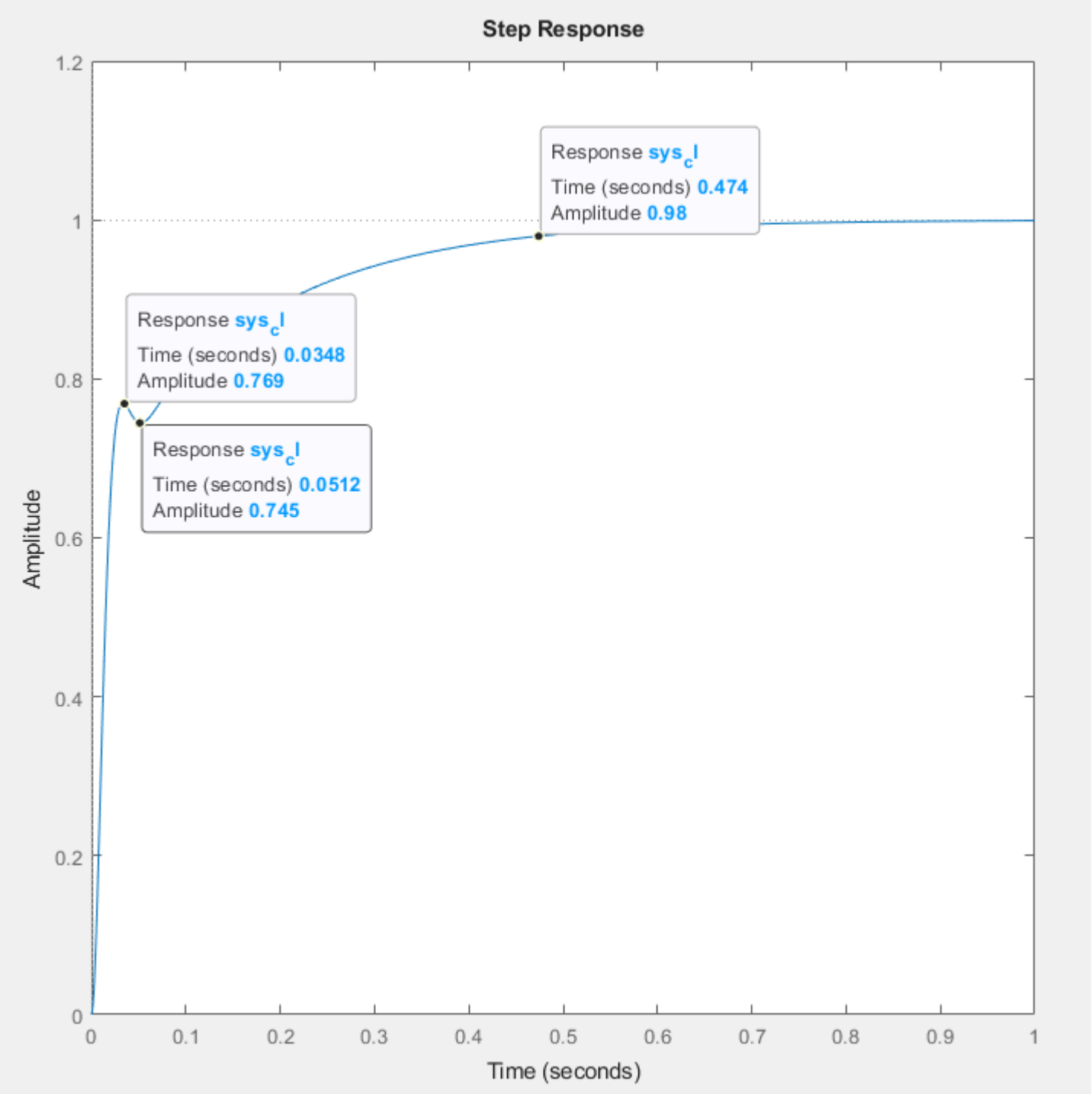

Figure 19. Matlab step response with PD compensating zero at -77.1 rad/s and gain K = 0.0173.

Table 7: PID Performance Comparison			
Parameter	**Target**	**Matlab Result**	**% Δ**
Poles	-76.67 ± j119.68	-77.1 ± j120	(Re) 0.56% / (Im) 0.27%
Tp	44.6ms	34.8ms	-24.69%
Ts	-	474ms	N/A
%OS	9.48%	0%	-9.48%
FV	1.0	1.0	0%

Table 7: PID performance comparison with percent differences.

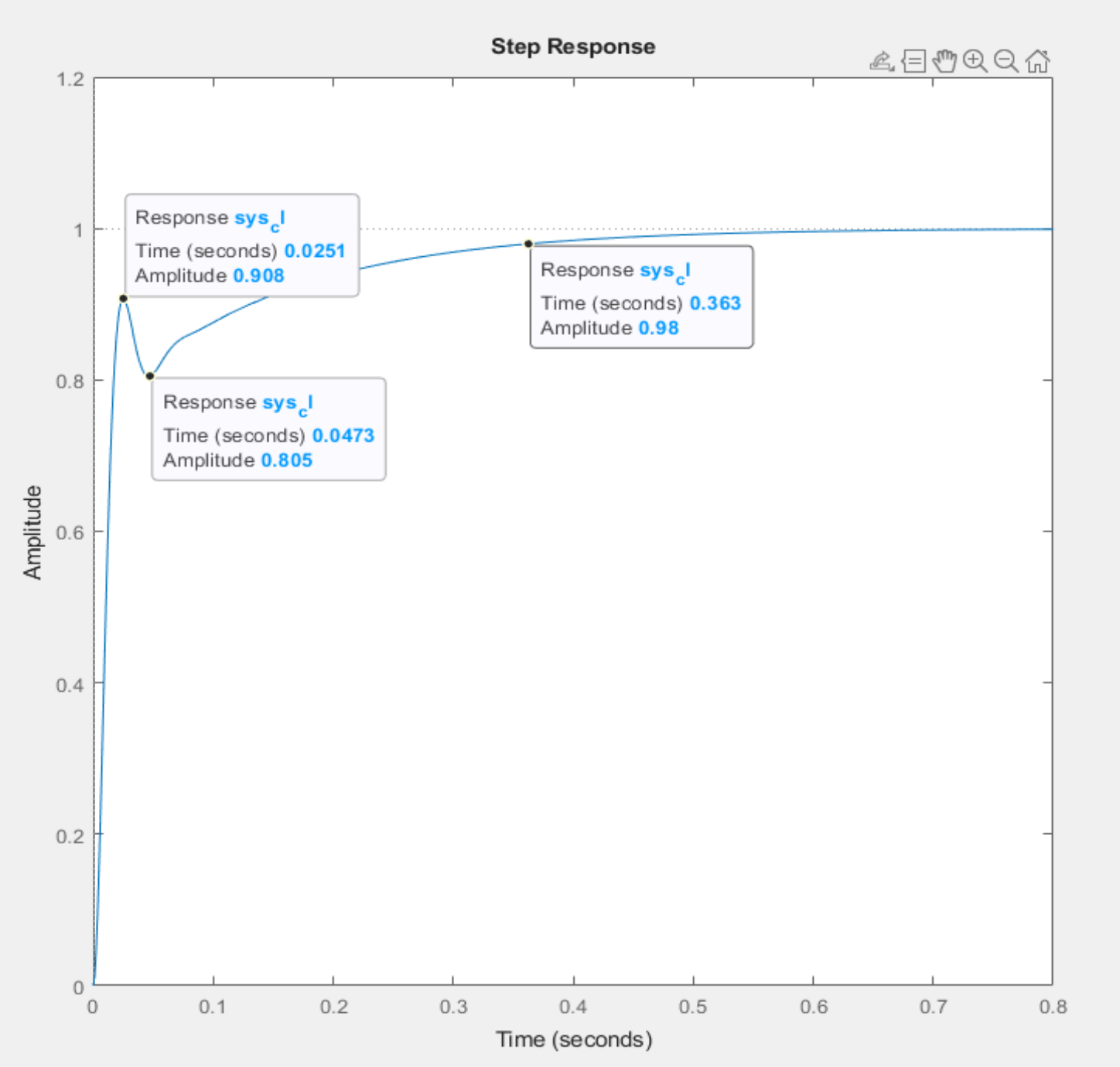

Figure 20. Matlab step response with PD compensating zero at -77.1 rad/s and gain K increased from 0.0173 to 0.0273

To determine the $G_{PID}(s)$ gain, the plant is separated and the numerator is multiplied out with gain K, as seen below in equations 15-17. The resultant gains are $K_D = 0.0273, K_P = 2.954, K_I = 24.37$ as shown in equation 17.

$$[15] \quad G_{PID}(s) = 0.0273\left(\frac{(s+9)(s+99.2)}{s}\right)$$

$$[15] \quad = 0.0273\left(\frac{s^2+108.2s+892.8}{s}\right)$$

$$[15] \quad = \frac{0.0273s^2+2.954s+24.37}{s}$$

$$[16] \quad G_{PID}(s) = \frac{0.0273s^2}{s} + \frac{2.954s}{s} + \frac{24.37}{s}$$

$$[17] \quad G_{PID}(s) = 0.0273s + 2.954 + \frac{24.37}{s}$$

Table 9: PID Performance Parameters			
Parameter	**Target**	**K = 0.0273**	**% Δ**
Tp	44.6ms	25.1ms	-55.95%
Ts	-	363ms	N/A
%OS	9.48%	0%	-9.48%
FV	1.0	1.0	0%

Table 9: PID performance parameters and the percent difference.

Section VI. Circuit Simulation of PID Controller

To determine the Rf value for the proportional part of the PID controller equation 18 was used, and as shown in figure 21 Rf corresponds to R10. For the capacitor of the integrator portion of the PID controller equation 19 is used and as shown in figure 21 C_I corresponds to C5. For the derivate channel of the PID controller equation 20 is used to determine the value and C_D corresponds to C4 in figure 21.

$$[18] \quad Rf = 2.954(1000\Omega) = 2{,}954\Omega$$

$$[19] \quad C_I = \frac{1}{(24.37)(1000\Omega)} = 41.03\mu F$$

$$[20] \quad C_D = \frac{0.0273}{(1000\Omega)} = 27.30\mu F$$

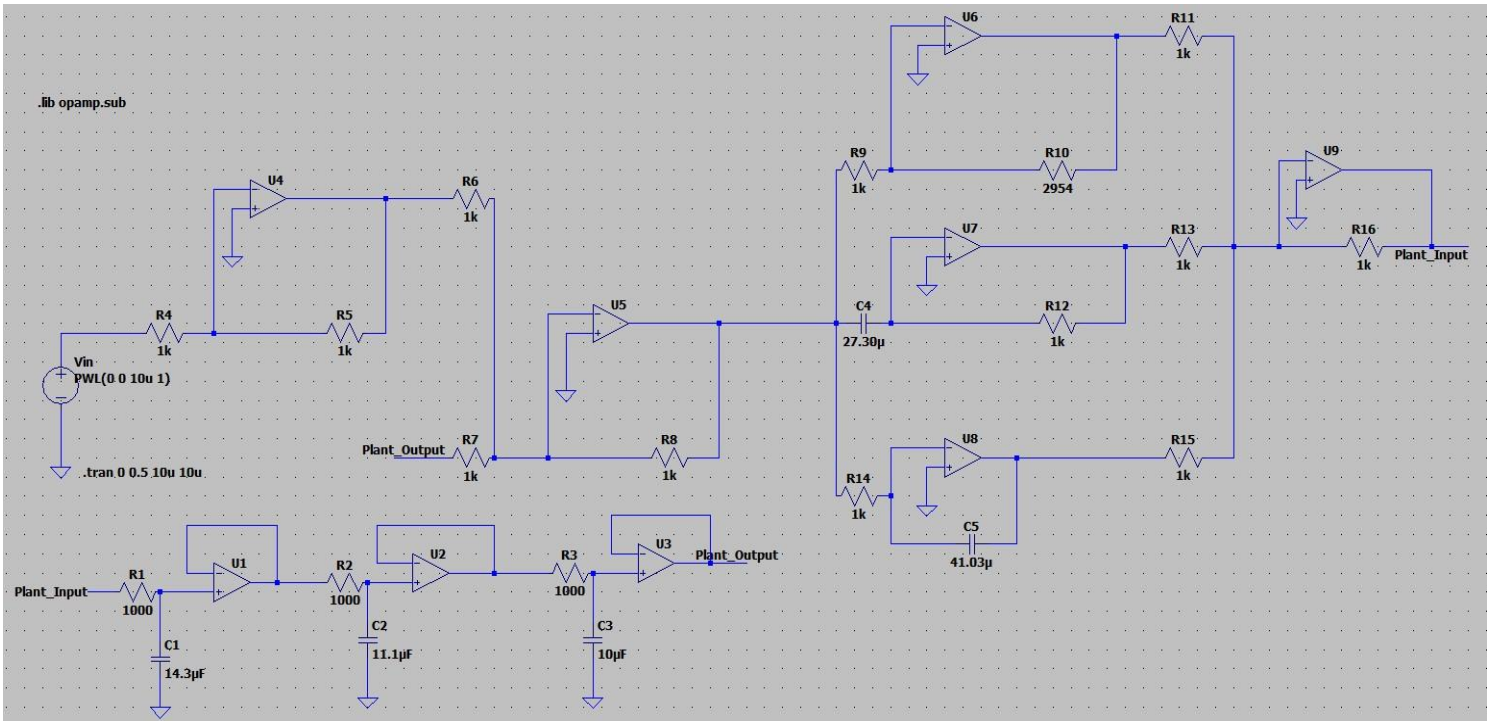

Figure 21. PID Controller, Plant, and Summing junction as captured in LTSpice circuit simulator.

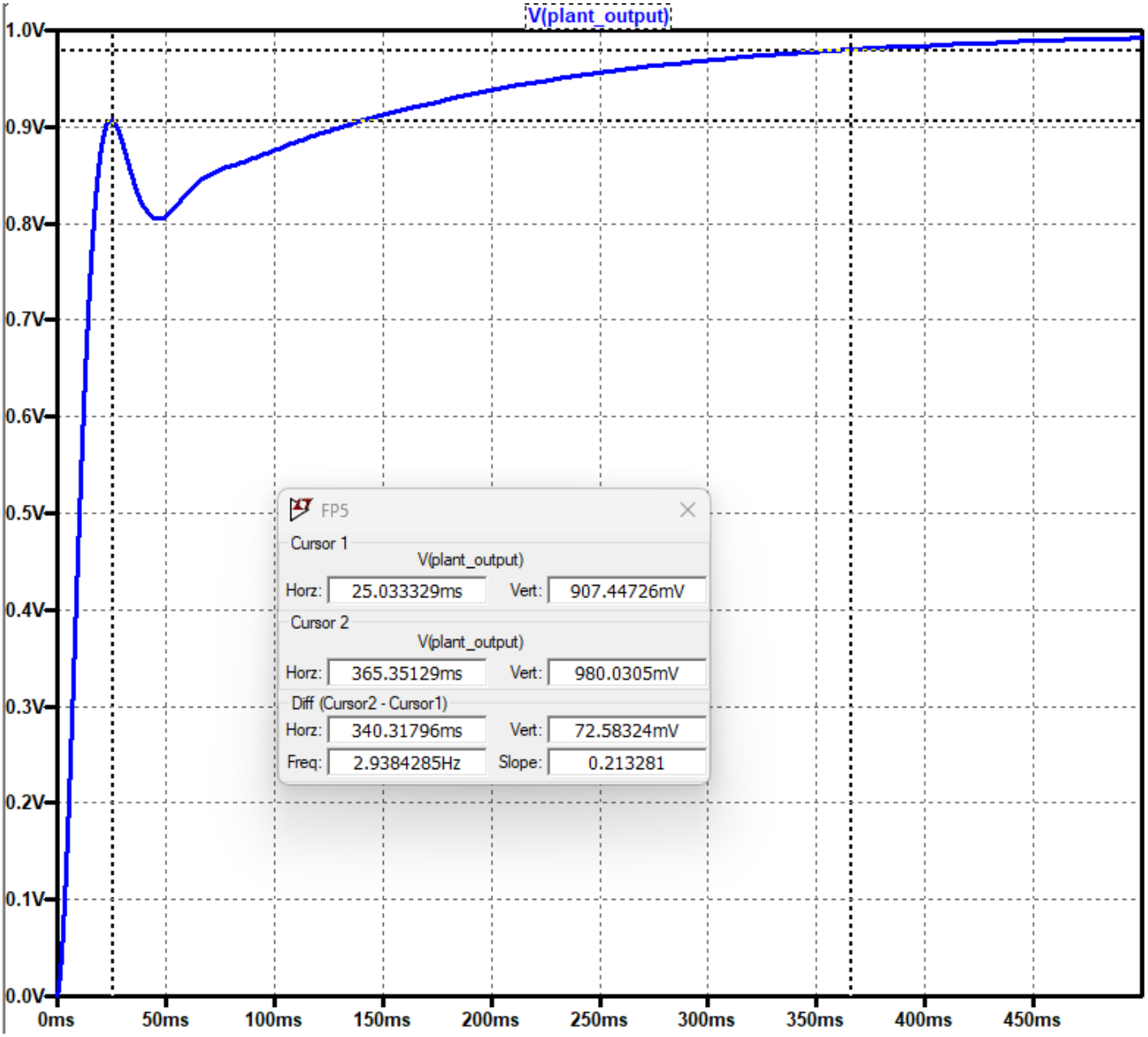

Figure 22. Complete system response in LTSpice circuit simulator.

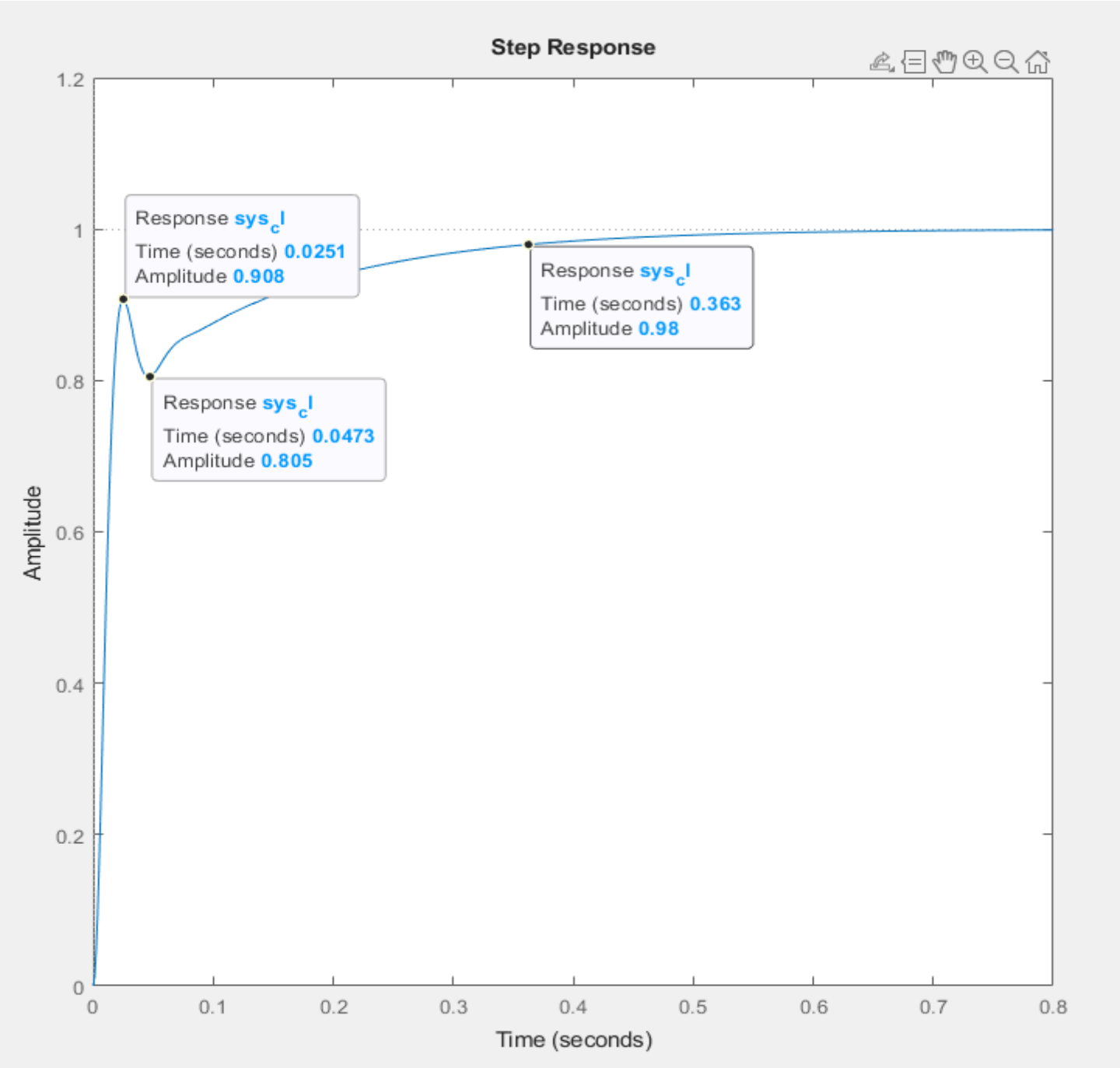

Figure 23. Matlab step response with PD compensating zero at -77.1 rad/s and gain K increased from 0.0173 to 0.0273.

Table 10: PID Performance Comparison Matlab vs. LTSpice				
Parameter	**Target**	**Matlab**	**LTSpice**	**% Δ**
Tp	44.6ms	25.1ms	25.03ms	0.28%
Ts	-	363ms	365.35ms	0.65%
%OS	≤ 9.48%	0%	0%	-
FV	-	1.0	1.0	-

Table 10. PID performance comparison for target vs Matlab vs LTSpice with percent differences between Matlab and LTSpice.

The PID controller is working as desired, the Tp target was not only met but almost 50% faster, and the percent difference between both Matlab and LTspice is 0.28% which is acceptable as it is less than 1%. The %OS is 0%, and the integrator is working correctly. Overall, the results are optimal for the PID controller's behavior.

Section VII. PID Tuning

In this section I have researched two different tuning algorithms, the Ziegler-Nichols PID method and the Skogestad's method. The goal is to apply both methods to my system and tune its performance without exceeding the percent overshoot maximum, and without degrading the time to peak (Tp) or the time to settle (Ts). My findings are shown below.

Ziegler-Nicholas (Z-N) Method

The Z-N method takes a heuristic approach to determine suitable values for the proportional (P), integral (I), and derivative (D) gains of a PID controller. Specifically, the proportional gain (Kp) is treated as the primary gain and serves as the foundation for the other gains (Ki and Kd). To start the integral gain (Ki) and derivative gain (Kd) are set to zero, with only the proportional gain (Kp) being adjusted. Thus, isolating the effect of the P channel's gain demonstrates the influence of the P channel on the system. Shown below is the root locus of the system with the D and I channel's gains set to zero, where the system is evaluated to find where it is semi stable.

First I changed the matlab code to have the plant separated from the PID controller with the gain K = 1 and verified the step response is the same as in the section prior in figure 23. The Matlab code is shown below with the changes and the output (figure 24) is as expected.

Matlab code:

```
s = tf('s');
Gplant = ((70)*(90)*(100))/((s+70)*(s+90)*(s+100));
GPID = (0.0273*s+2.954+24.37/s);
G = Gplant*GPID;
zeta = 0.54;
wn = 300;
K = 1;
sys_cl = feedback(K*G,1);
step(sys_cl,1);
axis([0 0.8 0 1.2]);
```

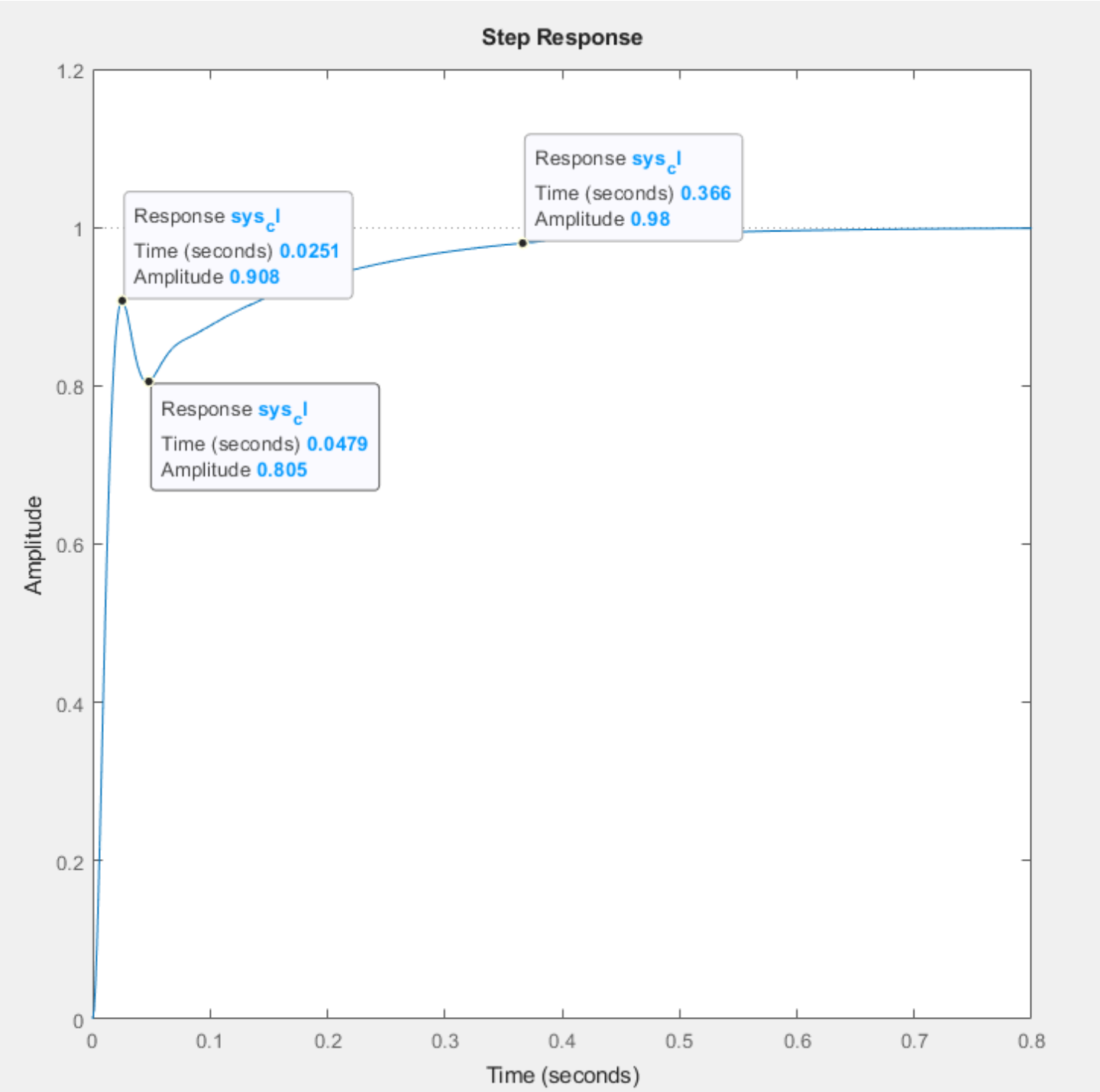

Figure 24. Simulated Matlab response with code adjustments confirming the step response is unchanged so the system is correct.

Now with the code adjusted with Ki and Kd set to zero, I can use the rlocus function to output the root locus to determine Ku (K ultimate or K max) using equation 21. This is shown below in figure 25.

$$[21] \quad K_u = K_P * 1.32 = 2.954 * 2.8 \ = 8.27$$

Matlab code:

```
s = tf('s');
Gplant = ((70)*(90)*(100))/((s+70)*(s+90)*(s+100));
% GPID = (0.0273*s+2.954+24.37/s); // For reference
GPID = (0*s+2.954+0/s);
G = Gplant*GPID;
zeta = 0.54;
wn = 300;
rlocus(G);
sgrid(zeta,wn);
axis([-120 5 -150 150]);
```

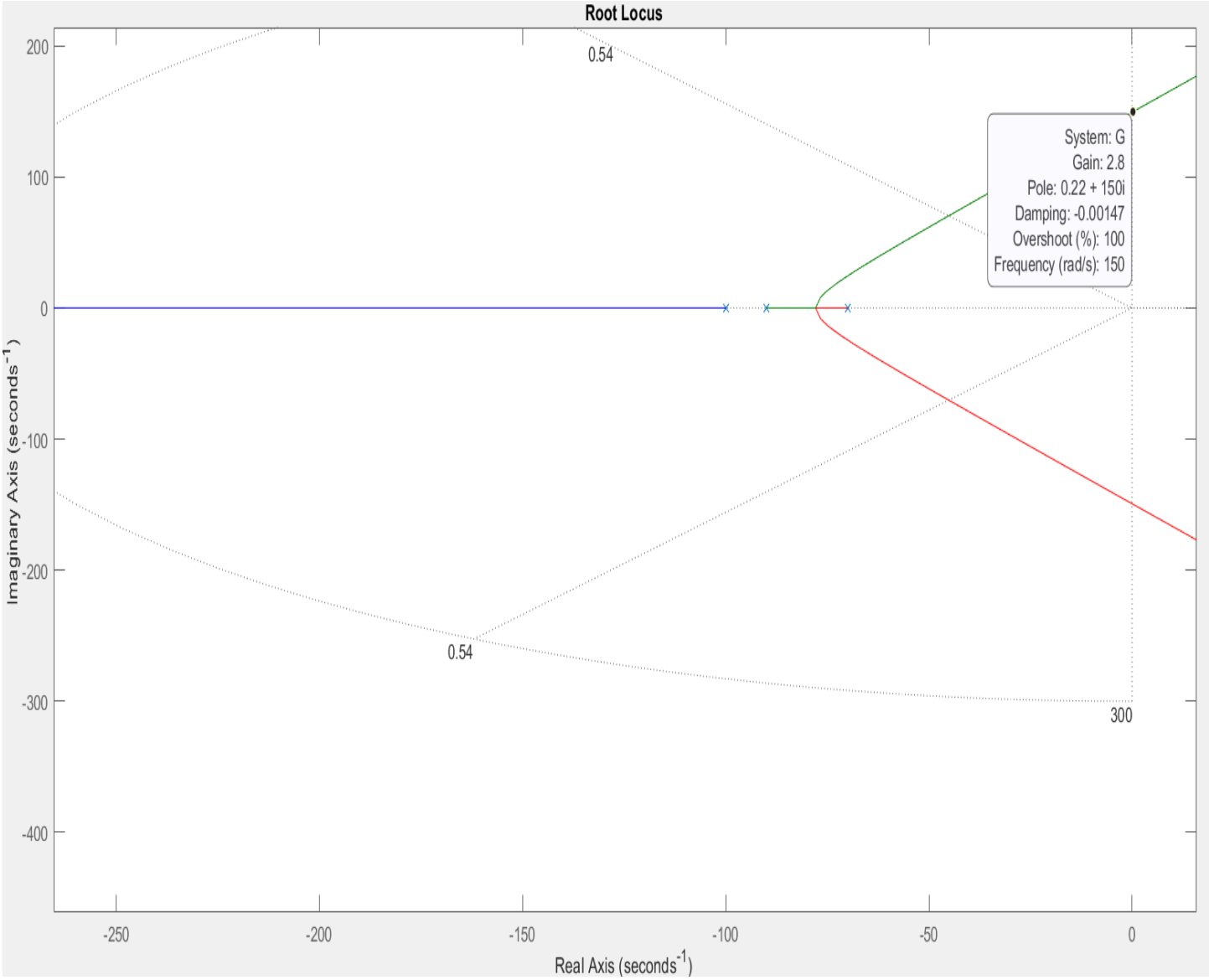

*Figure 25. Root locus to determine Ku gain value for the Z-N method. Ku = Kp * 2.8 = 2.954 * 2.8 = 8.27.*

Next I verified Ku = 8.27 by setting the Kp gain in matlab = Ku and confirm that the root locus now will intersect the imaginary axis at K = 1. It is as expected with K = 1.0 and frequency = 150 rad/s. This is shown below in figure 26.

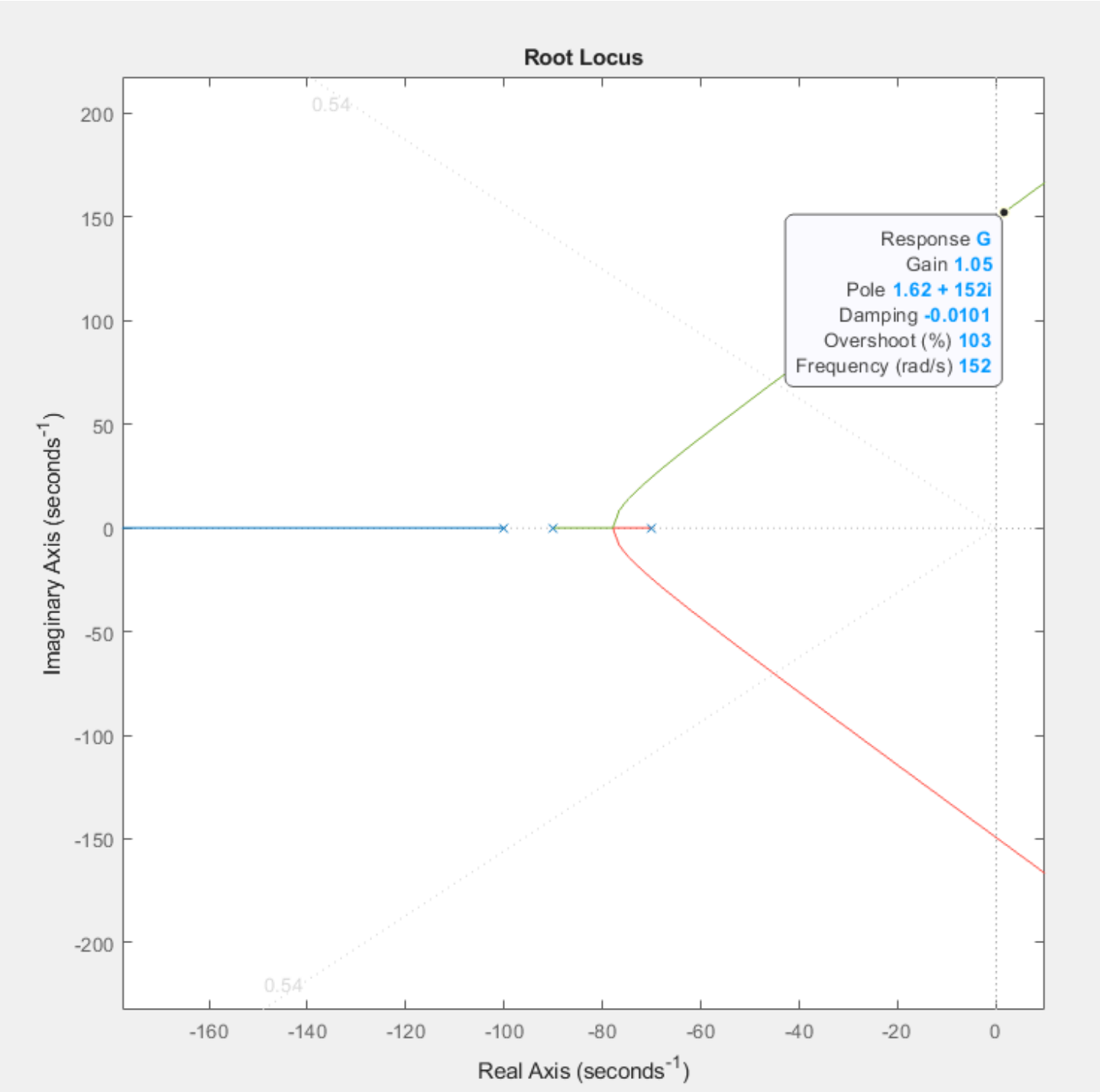

Figure 26. Verified Ku value root locus plot in Matlab with the frequency = 150 rad/s and the gain K at the imaginary axis = 1.

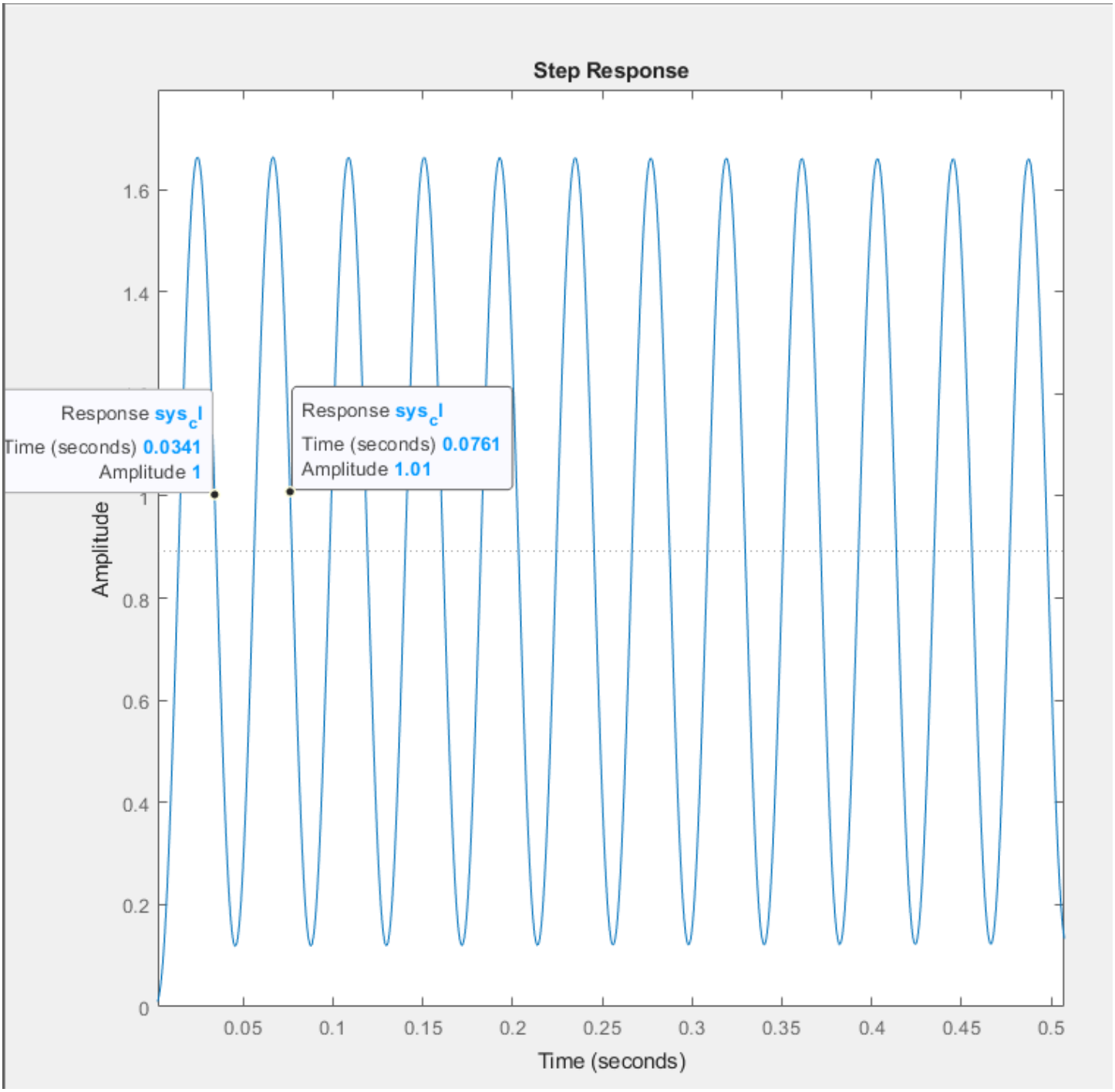

Figure 27. Matlab output verifying Ku = 8.27 is correct by showing steady oscillations with a step response input.

“Classic” Z-N Tuning Method:

The following equations are used to calculate the parameters to apply the “Classic” Z-N tuning to the PID.

$$[22] \quad T_u = 0.0761 - 0.0341 \ = 42ms$$
$$[23] \quad K_p = K_u * 0.6 = 8.27 * 0.6 \ = 4.96$$
$$[24] \quad K_i = 2K_P/T_u = 9.92/42ms \ = 236.19dB$$
$$[25] \quad K_d = (K_P * T_u)/8 = (4.96 * 42ms)/8 \ = 0.026dB$$

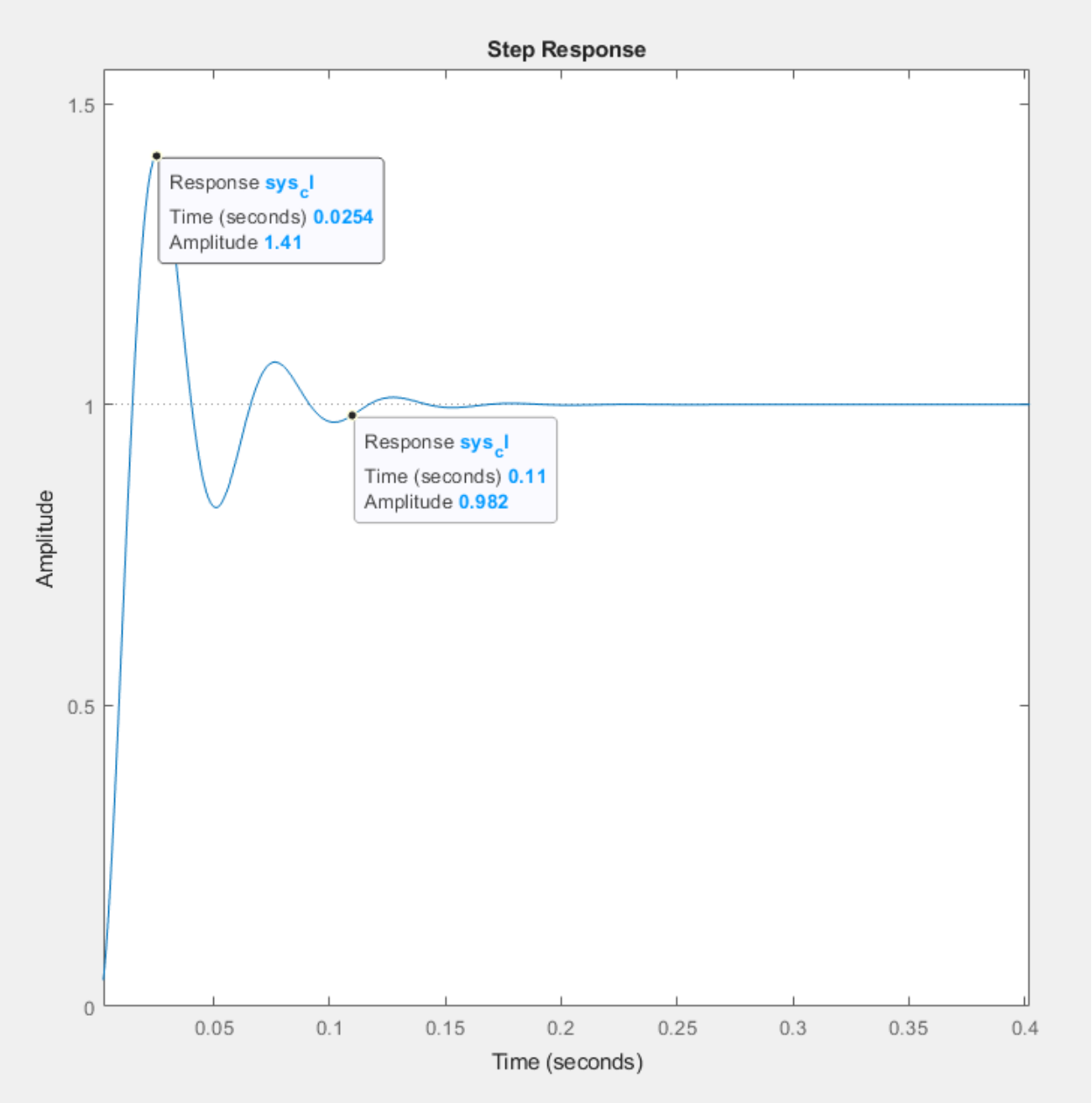

Figure 28. Step response for the closed loop system with classic Z-N tuning applied. Notice the %OS is 41% which is well above our desired max of 9.48% and the Tp and Ts have both decreased.

“Some Overshoot” Z-N Tuning Method:

The equations below are used to calculate the parameters for applying the “Some Overshoot” Z-N tuning method for PID controllers.

$$[26] \quad K_p = K_u * 0.33 = 8.27 * 0.33 = 2.73$$

$$[27] \quad K_i = 2K_P/T_u = 5.46/42ms \ = 130dB$$

$$[28] \quad K_d = (K_P * T_u)/3 = (2.07 * 42ms)/3 \ = 0.038dB$$

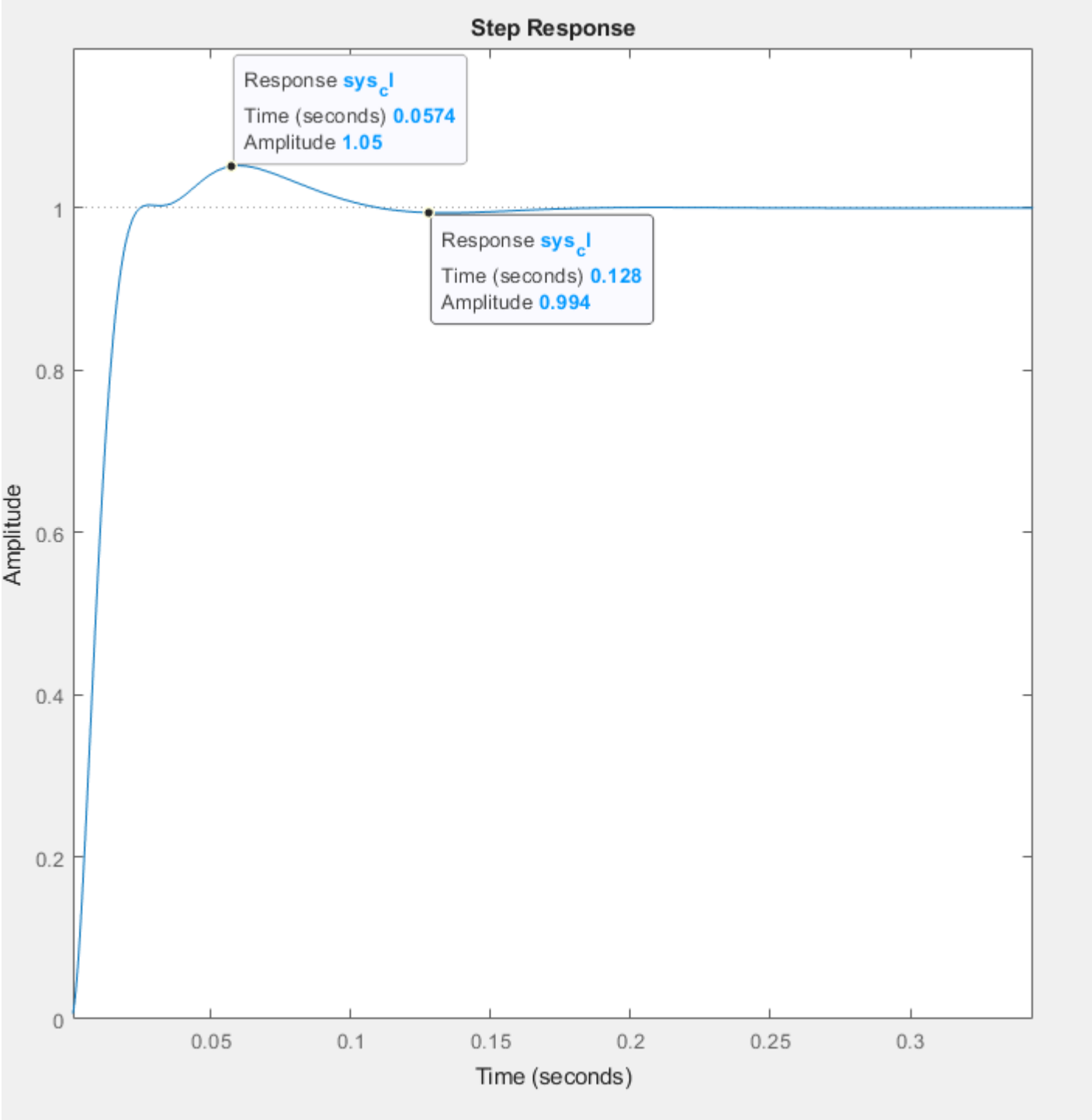

Figure 29. “Some Overshoot” tuning method applied to PID resulting with a much smaller %OS of 5% which is under our target and significantly slowing down the system compared to the “Classic” method, but the speed is too slow for our target.

Pessen Integral Z-N Tuning Method

The equations below are used to determine the parameters of the "Pessen Integral" Z-N PID tuning method.

$$[29] \quad K_p = K_u * 0.7 = 8.27 * 0.7 = 5.78$$

$$[30] \quad K_i = 2.5K_P/T_u = 5.46/42ms \ = 344.64dB$$

$$[31] \quad K_d = 0.15K_P * T_u = 0.867 * 42ms \ = 0.036dB$$

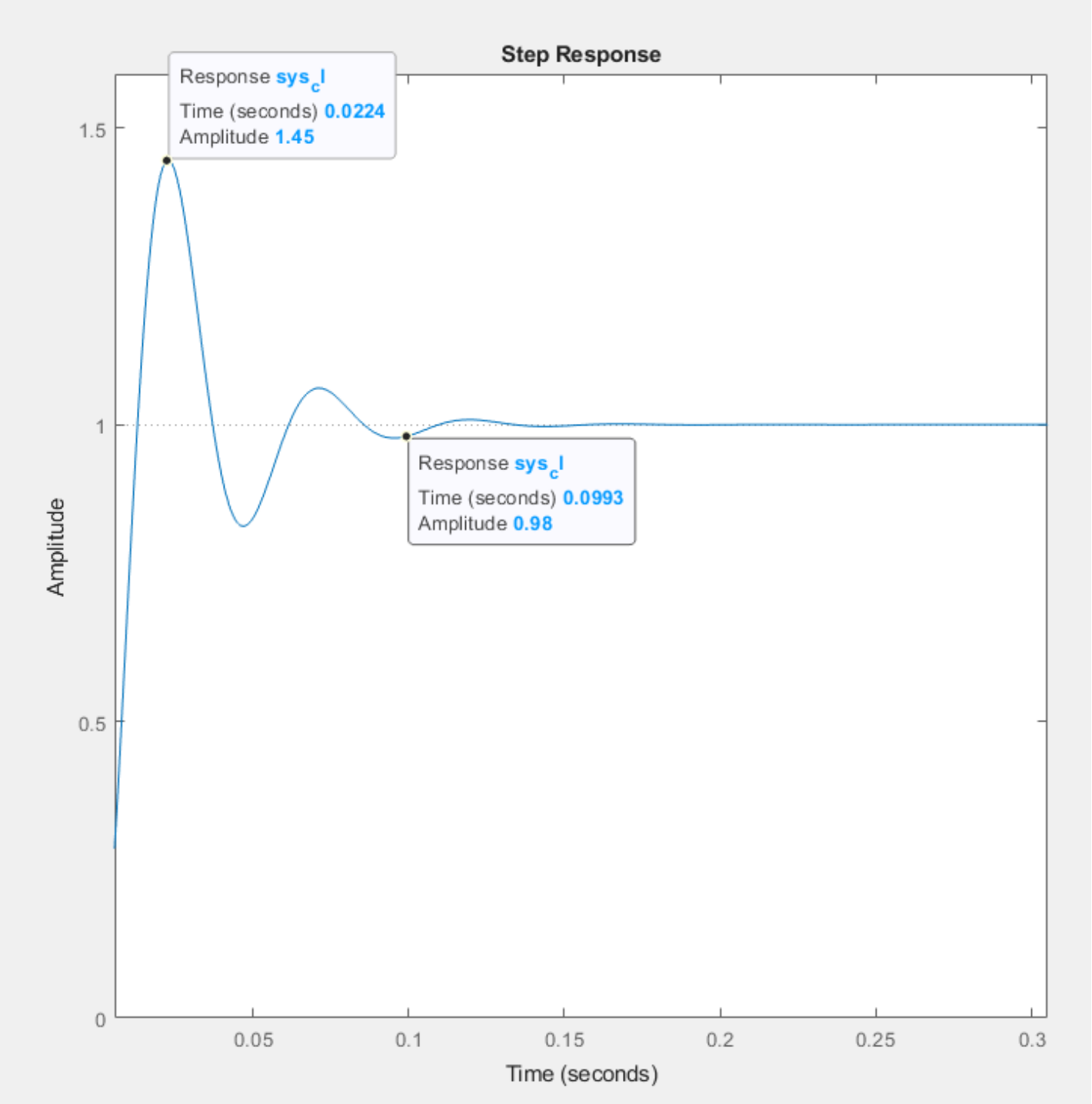

Figure 30. The output response with Pessen Integral Z-N tuning method applied. %OS is well above our tolerance and the Tp improves from the "Classic" method but the Ts is slower.

Custom Z-N Based Tuning Method

None of these Z-N tuning models fit perfectly in tuning the PID controller to match the desired targets of less than 9.48% OS and at least 50% improvement on Tp speed. However, after observing the behavior of the response simulation I have employed my own modified Z-N tuning method. I have noticed in general as Ki decreases the %OS decreases, as Kp increases the overall speed increases and %OS increases, and as Kd is adjusted oscillations increase before reaching the final value. With this in mind I have used the following equations to achieve a response that is within the target values.

$$[32] \quad K_p = K_u * 0.5 = 8.27 * 0.4 = 3.3$$

$$[33] \quad K_i = 1.25K_P/T_u = 6.6/42ms \ = 98.21dB$$

$$[34] \quad K_d = \frac{(K_P * T_u)}{4} = (3.3 * 42ms)/4 \ = 0.035dB$$

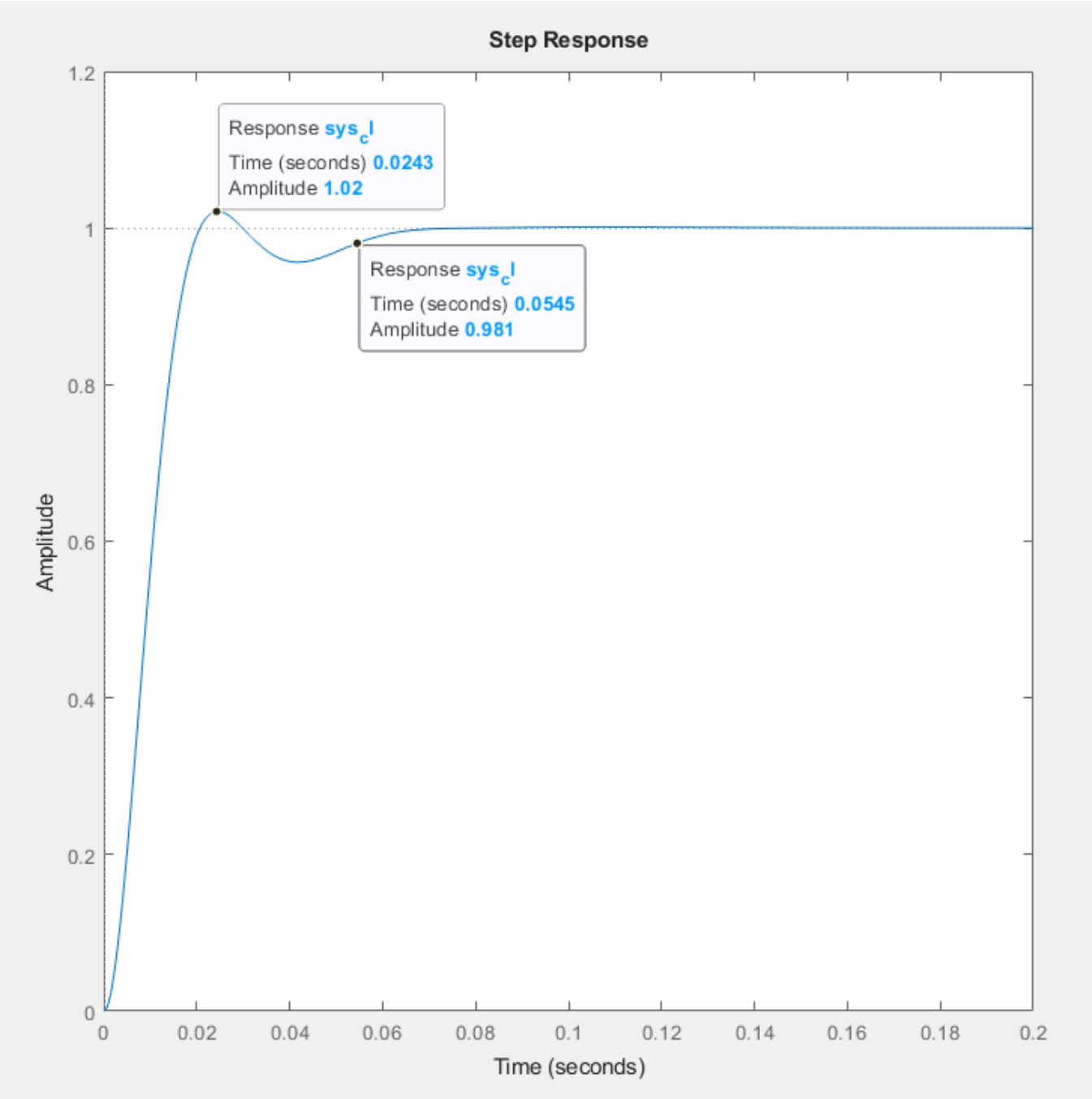

Figure 31. Matlab simulation result with Custom Z-N Tuning applied. Note that all targets are met with the %OS being 2%, the Tp at 24.3ms and the Ts at 54.5ms. Overall the system is faster and demonstrates less overshoot.

LTSpice vs Matlab Simulations for Custom Z-N Based Method:

To calculate the physical component values to implement the system in LTSpice the following equations are used:

$$[35] \quad R_P = 3.3 * 1K\Omega = 3.3K\Omega$$

$$[36] \quad C_i = (98.21 * 1K\Omega)^{-1} = 10.18\mu F$$

$$[37] \quad C_d = 0.035/1K\Omega = 35\mu F$$

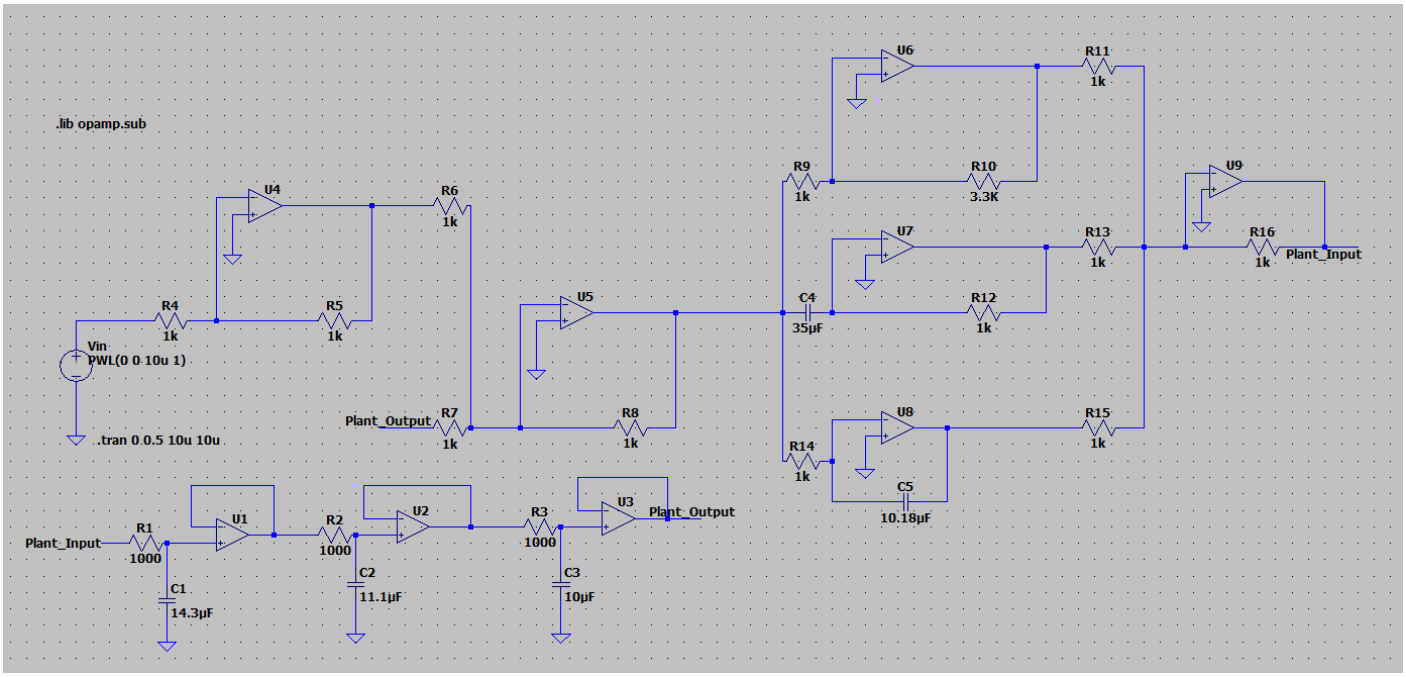

Figure 32. LTSpice schematic with adjusted component values for the PID controller shown on the right side of the schematic.

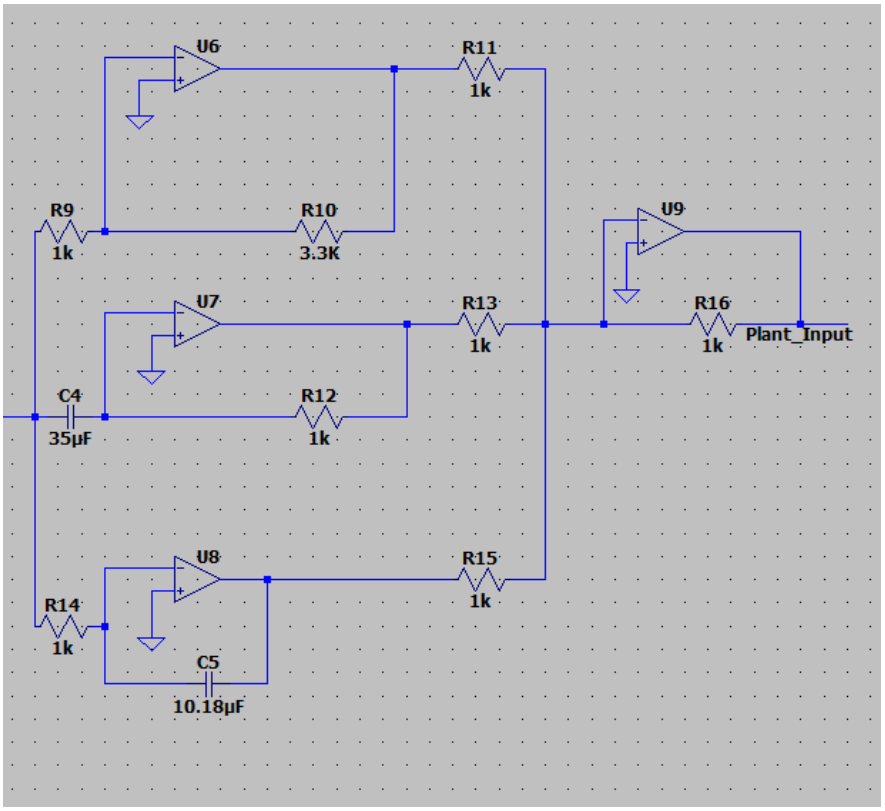

Figure 33. Zoom in on PID controller for system with updated values based on tuning parameters for R10, C4, and C5.

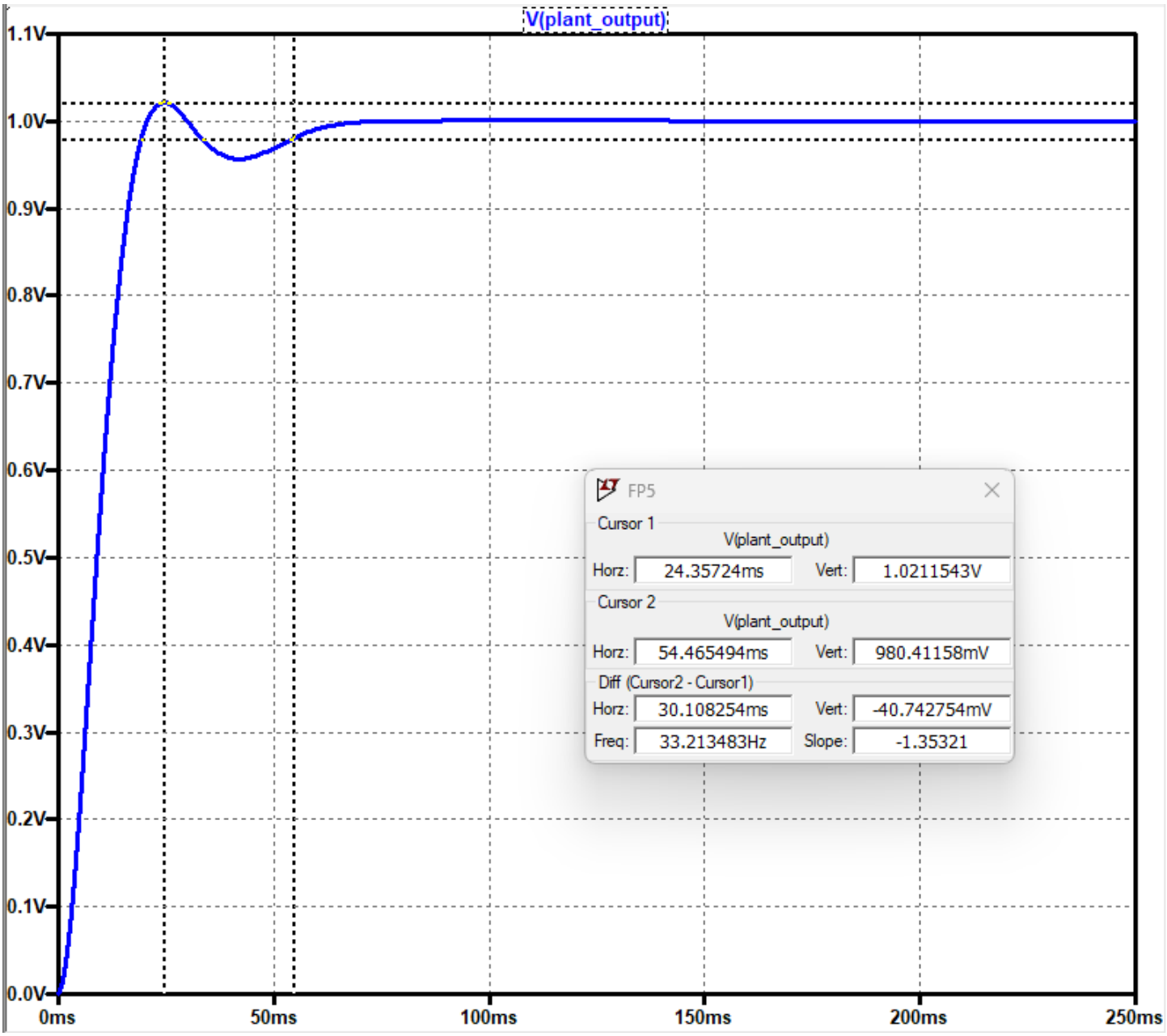

Figure 34. Step response with Custom Tuning applied with %OS at 2.1%, where cursor 1 placed at the peak and the second cursor placed at the settle point.

Table 11: PID Performance Comparison Tuned vs Untuned, Matlab vs LTSpice				
Parameter	**Target**	**Matlab**	**LTSpice**	**%Δ**
Tp	44.6ms	24.3ms	24.36ms	-58.93%
Ts	≤ 81.5ms	54.5ms	54.47ms	-39.70%
%OS	≤ 9.48%	2%	2.1%	-130.31%
FV	1.0	1.0	1.0	-

Table 11. Performance comparison for untuned vs “Some Overshoot” tuning method in Matlab vs LTSpice simulations.

Skogestad's Method

Skogestad's tuning method is a model-based PID tuning approach. The method is centered around approximating your process using a first-order plus dead time (FOPDT) model.

$$[38] \quad G(s) = \frac{K}{\tau s+1} e^{-\theta s}$$

Where:
K is the process gain

τ is the time constant

θ is the dead time (time before any change occurs)

Specifically for PID tuning parameters the following equations are implemented:

$$[39] \quad K_p = \frac{\tau}{\theta+\tau_c} * \frac{1}{K}$$

$$[40] \quad \tau_i = \min(\tau, 4(\theta + \tau_c)$$

$$[41] \quad K_i = \frac{K_p}{\tau_i}$$

$$[42] \quad \tau_d = \frac{\theta\tau}{\theta+\tau_c}$$

$$[43] \quad K_d = K_p * \tau_d$$

Where the parameters are:

K_p proportional gain

K_i integral gain

K_d derivative gain

τ_i integral time

τ_c desired closed-loop time constant

τ_d derivative time

In comparison to the Z-N method, this method isn't based on balancing performance with stability. The Skogestad method doesn't rely on trial-and-error or specific tables of formulas like most other methods, it is more analytical and customizable. The FOPTD model is used to calculate controller parameters based around the desired closed loop time constant. A smaller τ_c results in a faster response but potentially less robustness, while a larger τ_c slows down the system but increases in noise reduction and higher stability.

Section VIII. Discussion and Conclusion

1. What are the advantages and disadvantages of PID control?

PID controllers are widely used because they are simple, intuitive, and effective for many types of systems. The major advantage is their ability to improve system stability, reduce steady-state error, and provide faster and smoother responses without requiring a highly detailed model of the plant. However, they do have limitations. In systems that are non-linear, having more than 2 poles, have significant delays, or require advanced adaptability, causing the PID controllers to possibly struggle to maintain performance. Tuning can also be challenging, especially when trying to meet specific time-domain constraints like overshoot and rise time simultaneously.

2. Describe what you researched regarding PID tuning, and how you applied it to this project. What are some PID tuning methods? How effective were they for your project? What were your results?

For this project, I explored multiple PID tuning strategies including the Ziegler-Nichols (Z-N) method (Classic, Some Overshoot, and Pessen Integral variants), a Custom Z-N-based approach, and the Skogestad method. Each method was evaluated based on how well it met two key design targets: a 50% improvement in time to peak and a maximum overshoot of 9.48%.

The “Classic” Z-N method resulted in a fast response but significantly overshot the target with 41% overshoot. The “Some Overshoot” and Pessen methods improved the overshoot but did not meet the speed requirement. Skogestad’s method was useful as an analytical approach but required parameter estimation that was less practical for this application. Ultimately, I developed a custom tuning strategy using the Z-N approach that balanced all three PID gains and resulted in a time to peak of 24.3ms (a 58.93% improvement), a settling time of 54.5ms, and only 2.0% overshoot—exceeding the project’s performance goals.

3. Compare and contrast your Matlab vs. SPICE results.

Table 11: PID Performance Comparison Tuned vs Untuned, Matlab vs LTSpice				
Parameter	**Target**	**Matlab**	**LTSpice**	**%Δ**
Tp	44.6ms	24.3ms	24.36ms	-58.93%
Ts	≤ 81.5ms	54.5ms	54.47ms	-39.70%
%OS	≤ 9.48%	2%	2.1%	-130.31%
FV	1.0	1.0	1.0	-

Table 11. Performance comparison for untuned vs "Some Overshoot" tuning method in Matlab vs LTSpice simulations.

The Matlab and LTSpice simulations produced nearly identical performance results for the final tuned PID controller. As shown in Table 11:

- Time to peak: 24.3ms (Matlab) vs 24.36ms (LTSpice), 0.28% difference
- Settling time: 54.5ms vs 54.47ms, 0.05% difference
- Overshoot: 2.0% vs 2.1%, 0.1% difference
- Final value: 1.0 in both

These differences are negligible and likely due to minor numerical precision variations or simulation resolution settings. Both tools validated the same system behavior, but Matlab provided more flexibility in analyzing and adjusting the system analytically, while LTSpice was essential for verifying component-level circuit performance. For accuracy in physical implementation, LTSpice is likely more realistic due to its detailed analog modeling.

4. How do you believe that a digital implementation of a PID controller may differ from this analog implementation?

A digital PID controller would be implemented in software using discrete-time approximations of the PID algorithm. Unlike analog controllers, digital implementations require sampling, quantization, and can introduce delays due to computation or ADC/DAC processes. I think a digital PID controller could be used in more creative implementations to solve problems than an analog controller, as there is more control over the parameters and flexibility in how to use them. The output from a digital PID controller is has more precision because you don't have to take into consideration the physical tolerances and behaviors of non-ideal components. However, you do have to keep in mind aliasing and computational power when determining the sample frequency.

5. What did you learn from this Final Project?

I have learned a lot from this project, there are many new skills and theoretical knowledge that I have obtained, as these were the intended results of completing this project. However, I also gained unexpected insights and skills that have manifested in useful ways in my other courses.

For the intended skills, I now have hands-on experience in designing and analyzing PID controllers from both a theoretical and practical perspective. I learned how to derive and verify transfer functions, apply root locus and frequency response techniques, and tune controllers using multiple strategies. I have obtained proficiency in practical skills in using Matlab for mathematical modeling and LTSpice for physical circuit simulations and validation.

The unexpected insights I have gained are more conceptual, like observing systems and how they behave. For example, I have used state space representations as a basis to model a problem as a sort of non-traditional system, allowing optimization of code algorithms and creative approaches to solve problems.